

# MONEY CARD PLATFORM — M0 V10 COMPLETE API CONTRACT

## Developer 2 Authoritative Backend Implementation Specification

**Document Version:** 10.0.0 (Frozen V1 Handover)

**Source of Truth:** M0\_Updated\_V10\_Developer2\_Complete\_API\_Contract.pdf

**Target Backend Stack:** Node.js • Express • PostgreSQL • Prisma ORM • JWT Authentication

### Contract Verification Status

- 65 API endpoints documented
- 65 Postman requests configured & verified
- 58 / 58 automated contract assertions passed
- 100% schema parity for executed assertions
- 0 missing endpoints

**Verification Date:** August 2026

---

## 2. Document Purpose

This document is the **authoritative, implementation-ready contract handoff** for Developer 2. It contains the verbatim Request and Response JSON specifications for every single API endpoint defined in the Money Card platform.

Developer 2 must implement the real backend so that:

- 1. Every route returns the exact JSON keys and envelope structures specified here.
- 2. Every HTTP status code matches the tested contracts.
- 3. The React Web Frontend and Flutter Mobile Apps can seamlessly switch from Mock API to the real backend without a single code change.

## 3. M0 V10 Frozen System Rules & Removals

Per M0 V10 specifications, the following architectural invariants are permanently frozen:

| Rule / Invariant                    | Status                        | Backend Implementation Mandate                                                                                                        |            |              |
|-------------------------------------|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|------------|--------------|
| <code>Plan.transaction_limit</code> | ■ REMOVED                     | Plans do NOT contain transaction quota or limit fields. Transactions are unlimited per tier.                                          |            |              |
| Monthly Transaction Count           | ■ ANALYTICS ONLY              | Transaction count is aggregated on-the-fly for reporting only. It must NEVER block card transactions.                                 |            |              |
| <code>Product.tags</code>           | ■ REMOVED                     | Tags field is completely removed from DB, CSV import, and APIs.                                                                       |            |              |
| <code>Product.category</code>       | ■■ MULTI-SELECT               | Must be an array of strings ( <code>string[]</code> ). In CSV, delimited with pipe <code>`</code> .                                   | (e.g. Veg` | Fast Food`). |
| CSV Import Columns                  | ■ 3 COLUMNS ONLY              | Exactly <code>itemName</code> , <code>category</code> , <code>price</code> . Selected branch context is sent in request headers/body. |            |              |
| Subscription Billing                | ■■ DIRECT / OFFLINE           | Direct bank transfer and Super Admin review workflow. No payment gateway webhooks.                                                    |            |              |
| Analytics Export                    | ■ PDF ONLY                    | <code>GET /api/v1/analytics/export?format=pdf</code> returns PDF. No CSV/Excel exports.                                               |            |              |
| Formal Reports                      | ■ PDF ONLY                    | <code>GET /api/v1/reports/:id/pdf</code> returns PDF download only.                                                                   |            |              |
| Card State Machine                  | ■ AVAILABLE ↔ ACTIVE, BLOCKED | Available cards transition to Active on session start. Return refunds balance and resets to Available.                                |            |              |

## 4. API Architecture

- **Base URL:** `/api/v1`
- **Protocol:** HTTPS (HTTP for local dev at `http://localhost:4000/api/v1`)
- **Data Format:** `application/json` (UTF-8) for all requests and responses, except binary PDF downloads (`application/pdf`).
- **Multi-Tenancy:** Organization and Branch boundary isolation strictly enforced on every non-public route.

## 5. Authentication Rules

- **Header:** `Authorization: Bearer <JWT_ACCESS_TOKEN>`
- **Access Token Expiry:** 15 minutes (short-lived).

- **Refresh Token:** Stored in HTTP-only secure cookie or body; rotated upon [/api/v1/auth/refresh](#).
  - **Public Endpoints:** [/auth/login](#), [/auth/refresh](#), [/auth/forgot-password](#), [/auth/reset-password](#), and all [/public/\\*](#) routes require no Bearer header.
- 

## 6. Standard JSON Envelopes

Every response from the backend MUST use one of the following standard envelopes:

### Standard Success Envelope

```
{
  "success": true,
  "data": {
    "id": "rec_001",
    "name": "Example Item"
  }
}
```

### Standard Paginated Success Envelope

```
{
  "success": true,
  "data": {
    "items": [ ... ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 150,
      "totalPages": 8
    }
  }
}
```

### Standard Error Envelope

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human readable and actionable error message."
  }
}
```

---

## 7. Global Error Contract (21 Standard Codes)

| Error Code                         | HTTP Status | Meaning & Occurrence Scenario                                                                   |
|------------------------------------|-------------|-------------------------------------------------------------------------------------------------|
| <b>VALIDATION_ERROR</b>            | 400         | Invalid input, parameters, headers, or state-independent format failures.                       |
| <b>CSV_VALIDATION_ERROR</b>        | 400         | Invalid CSV file format, header mismatch, duplicate rows, or malformed data lines.              |
| <b>UNAUTHORIZED</b>                | 401         | Missing, invalid, or expired authentication credentials.                                        |
| <b>FORBIDDEN</b>                   | 403         | Authenticated identity lacks general required role or administrative privilege.                 |
| <b>PERMISSION_DENIED</b>           | 403         | Authenticated user lacks the specific required M0 operational permission code.                  |
| <b>ORGANIZATION_ACCESS_DENIED</b>  | 403         | Resource belongs to another tenant organization (Multi-tenant isolation barrier).               |
| <b>BRANCH_ACCESS_DENIED</b>        | 403         | Staff member is not assigned to the requested branch.                                           |
| <b>NOT_FOUND</b>                   | 404         | Generic resource does not exist or is intentionally hidden.                                     |
| <b>CARD_NOT_FOUND</b>              | 404         | Card ID or QR token does not resolve to any physical card in current scope.                     |
| <b>SESSION_NOT_FOUND</b>           | 404         | Requested Card Session ID was not found.                                                        |
| <b>SUBSCRIPTION_NOT_FOUND</b>      | 404         | Requested organization subscription record was not found.                                       |
| <b>CARD_NOT_AVAILABLE</b>          | 409         | Attempted to issue/start a session with a card not in AVAILABLE state (e.g. ACTIVE or BLOCKED). |
| <b>ALREADY_SETTLED</b>             | 409         | Attempted operation on a Card Session that has already been returned or settled.                |
| <b>REFUND_ALREADY_PROCESSED</b>    | 409         | Attempted to double-refund a transaction or settled session.                                    |
| <b>DUPLICATE_REQUEST</b>           | 409         | Idempotency violation or duplicate resource creation request.                                   |
| <b>PLAN_LIMIT_REACHED</b>          | 409         | Action blocked because current organization reached effective staff, branch, or card limits.    |
| <b>DOWNGRADE_LIMIT_EXCEEDED</b>    | 409         | Plan downgrade cannot be applied because current usage exceeds lower plan limits.               |
| <b>PORTAL_SESSION_EXPIRED</b>      | 410         | User Portal temporary session token has expired or the underlying card session was settled.     |
| <b>INSUFFICIENT_BALANCE</b>        | 422         | Card session balance is lower than purchase amount. Authoritative server check failed.          |
| <b>INSUFFICIENT_INVENTORY</b>      | 422         | Available inventory stock is insufficient to fulfill purchase quantity or stock adjustment.     |
| <b>PAYMENT_VERIFICATION_FAILED</b> | 422         | Manual UPI or payment reference verification failed during session recharge.                    |
| <b>INVALID_BUSINESS_STATE</b>      | 422         | Operation violates M0 business rules or state machine constraints.                              |
| <b>INTERNAL_SERVER_ERROR</b>       | 500         | Unexpected server failure or unhandled exception.                                               |

## 8. Complete Endpoint Index (65 Routes)

| #  | Method | Endpoint Path                                   | Auth       | Roles Allowed                 | Permission Required | Scope               |
|----|--------|-------------------------------------------------|------------|-------------------------------|---------------------|---------------------|
| 1  | POST   | /api/v1/auth/login                              | PUBLIC     | SUPER_ADMIN, ORG_ADMIN, STAFF | NONE                | Global              |
| 2  | POST   | /api/v1/auth/refresh                            | PUBLIC     | SUPER_ADMIN, ORG_ADMIN, STAFF | NONE                | Global              |
| 3  | POST   | /api/v1/auth/logout                             | BEARER JWT | SUPER_ADMIN, ORG_ADMIN, STAFF | NONE                | Self                |
| 4  | GET    | /api/v1/auth/me                                 | BEARER JWT | SUPER_ADMIN, ORG_ADMIN, STAFF | NONE                | Self                |
| 5  | POST   | /api/v1/auth/forgot-password                    | PUBLIC     | PUBLIC                        | NONE                | Global              |
| 6  | POST   | /api/v1/auth/reset-password                     | PUBLIC     | PUBLIC                        | NONE                | Global              |
| 7  | POST   | /api/v1/auth/change-password                    | BEARER JWT | SUPER_ADMIN, ORG_ADMIN, STAFF | NONE                | Self                |
| 8  | GET    | /api/v1/admin/organizations                     | BEARER JWT | SUPER_ADMIN                   | NONE                | Platform-wide       |
| 9  | POST   | /api/v1/admin/organizations                     | BEARER JWT | SUPER_ADMIN                   | NONE                | Platform-wide       |
| 10 | GET    | /api/v1/admin/organizations/:id                 | BEARER JWT | SUPER_ADMIN                   | NONE                | Target Org          |
| 11 | PATCH  | /api/v1/admin/organizations/:id                 | BEARER JWT | SUPER_ADMIN                   | NONE                | Target Org          |
| 12 | GET    | /api/v1/admin/plans                             | BEARER JWT | SUPER_ADMIN, ORG_ADMIN        | NONE                | Platform-wide       |
| 13 | POST   | /api/v1/admin/plans                             | BEARER JWT | SUPER_ADMIN                   | NONE                | Platform-wide       |
| 14 | GET    | /api/v1/admin/plans/:id                         | BEARER JWT | SUPER_ADMIN, ORG_ADMIN        | NONE                | Platform-wide       |
| 15 | PATCH  | /api/v1/admin/plans/:id                         | BEARER JWT | SUPER_ADMIN                   | NONE                | Platform-wide       |
| 16 | GET    | /api/v1/admin/organizations/:id/subscription    | BEARER JWT | SUPER_ADMIN                   | NONE                | Target Org          |
| 17 | PATCH  | /api/v1/admin/organizations/:id/subscription    | BEARER JWT | SUPER_ADMIN                   | NONE                | Target Org          |
| 18 | GET    | /api/v1/admin/organizations/:id/limit-overrides | BEARER JWT | SUPER_ADMIN                   | NONE                | Target Org          |
| 19 | PATCH  | /api/v1/admin/organizations/:id/limit-overrides | BEARER JWT | SUPER_ADMIN                   | NONE                | Target Org          |
| 20 | GET    | /api/v1/admin/subscriptions                     | BEARER JWT | SUPER_ADMIN                   | NONE                | Platform-wide       |
| 21 | GET    | /api/v1/admin/subscriptions/:id                 | BEARER JWT | SUPER_ADMIN                   | NONE                | Target Subscription |

|    |       |                                        |            |                               |               |                    |
|----|-------|----------------------------------------|------------|-------------------------------|---------------|--------------------|
| 22 | GET   | /api/v1/admin/subscription-payments    | BEARER JWT | SUPER_ADMIN                   | NONE          | Platform-wide      |
| 23 | GET   | /api/v1/admin/plan-change-requests     | BEARER JWT | SUPER_ADMIN                   | NONE          | Platform-wide      |
| 24 | GET   | /api/v1/admin/plan-change-requests/:id | BEARER JWT | SUPER_ADMIN                   | NONE          | Target Request     |
| 25 | PATCH | /api/v1/admin/plan-change-requests/:id | BEARER JWT | SUPER_ADMIN                   | NONE          | Target Request     |
| 26 | GET   | /api/v1/organization                   | BEARER JWT | ORG_ADMIN, SUPER_ADMIN        | NONE          | Own Org            |
| 27 | PATCH | /api/v1/organization                   | BEARER JWT | ORG_ADMIN, SUPER_ADMIN        | NONE          | Own Org            |
| 28 | GET   | /api/v1/branches                       | BEARER JWT | SUPER_ADMIN, ORG_ADMIN, STAFF | BRANCH_VIEW   | Own Org / Assigned |
| 29 | POST  | /api/v1/branches                       | BEARER JWT | ORG_ADMIN, SUPER_ADMIN        | BRANCH_MANAGE | Own Org            |
| 30 | GET   | /api/v1/branches/:id                   | BEARER JWT | SUPER_ADMIN, ORG_ADMIN, STAFF | BRANCH_VIEW   | Target Branch      |
| 31 | PATCH | /api/v1/branches/:id                   | BEARER JWT | ORG_ADMIN, SUPER_ADMIN        | BRANCH_MANAGE | Target Branch      |
| 32 | GET   | /api/v1/staff                          | BEARER JWT | ORG_ADMIN, SUPER_ADMIN        | STAFF_VIEW    | Own Org            |
| 33 | POST  | /api/v1/staff                          | BEARER JWT | ORG_ADMIN, SUPER_ADMIN        | STAFF_MANAGE  | Own Org            |
| 34 | PUT   | /api/v1/staff/:id/branches             | BEARER JWT | ORG_ADMIN, SUPER_ADMIN        | STAFF_MANAGE  | Target Staff       |
| 35 | PUT   | /api/v1/staff/:id/permissions          | BEARER JWT | ORG_ADMIN, SUPER_ADMIN        | STAFF_MANAGE  | Target Staff       |
| 36 | GET   | /api/v1/permissions                    | BEARER JWT | SUPER_ADMIN, ORG_ADMIN, STAFF | NONE          | Global             |
| 37 | GET   | /api/v1/cards                          | BEARER JWT | SUPER_ADMIN, ORG_ADMIN, STAFF | CARD_VIEW     | Own Org            |
| 38 | POST  | /api/v1/cards                          | BEARER JWT | ORG_ADMIN, STAFF              | CARD_ISSUE    | Own Org / Branch   |
| 39 | POST  | /api/v1/cards/resolve                  | BEARER JWT | ORG_ADMIN, STAFF              | CARD_VIEW     | Own Org            |
| 40 | POST  | /api/v1/cards/:id/block                | BEARER JWT | ORG_ADMIN, STAFF              | CARD_BLOCK    | Target Card        |
| 41 | POST  | /api/v1/cards/:id/unblock              | BEARER JWT | ORG_ADMIN, STAFF              | CARD_UNBLOCK  | Target Card        |
| 42 | POST  | /api/v1/card-sessions                  | BEARER JWT | STAFF, ORG_ADMIN              | CARD_ISSUE    | Branch Scoped      |
| 43 | POST  | /api/v1/card-sessions/:id/recharge     | BEARER JWT | STAFF, ORG_ADMIN              | RECHARGE      | Session Scoped     |
| 44 | POST  | /api/v1/card-sessions/:id/purchase     | BEARER JWT | STAFF, ORG_ADMIN              | PURCHASE      | Session Scoped     |
| 45 | POST  | /api/v1/card-sessions/:id/return       | BEARER JWT | STAFF, ORG_ADMIN              | CARD_RETURN   | Session Scoped     |

|    |       |                                                    |                        |                                     |                  |                     |
|----|-------|----------------------------------------------------|------------------------|-------------------------------------|------------------|---------------------|
| 46 | GET   | /api/v1/products                                   | BEARER JWT             | SUPER_ADMIN,<br>ORG_ADMIN,<br>STAFF | PRODUCT_VIEW     | Branch Scoped       |
| 47 | POST  | /api/v1/products                                   | BEARER JWT             | ORG_ADMIN,<br>STAFF                 | PRODUCT_MANAGE   | Branch Scoped       |
| 48 | GET   | /api/v1/inventory                                  | BEARER JWT             | SUPER_ADMIN,<br>ORG_ADMIN,<br>STAFF | INVENTORY_VIEW   | Branch Scoped       |
| 49 | PATCH | /api/v1/inventory/:id                              | BEARER JWT             | ORG_ADMIN,<br>STAFF                 | INVENTORY_MANAGE | Item Scoped         |
| 50 | GET   | /api/v1/inventory/import/template                  | BEARER JWT             | ORG_ADMIN,<br>STAFF                 | INVENTORY_IMPORT | Global              |
| 51 | POST  | /api/v1/inventory/import (Preview)                 | BEARER JWT             | ORG_ADMIN,<br>STAFF                 | INVENTORY_IMPORT | Branch Scoped       |
| 52 | POST  | /api/v1/inventory/import (Commit)                  | BEARER JWT             | ORG_ADMIN,<br>STAFF                 | INVENTORY_IMPORT | Branch Scoped       |
| 53 | GET   | /api/v1/analytics                                  | BEARER JWT             | SUPER_ADMIN,<br>ORG_ADMIN,<br>STAFF | VIEW_ANALYTICS   | Org / Branch Scoped |
| 54 | GET   | /api/v1/analytics/export                           | BEARER JWT             | SUPER_ADMIN,<br>ORG_ADMIN,<br>STAFF | VIEW_ANALYTICS   | Org / Branch Scoped |
| 55 | GET   | /api/v1/reports                                    | BEARER JWT             | SUPER_ADMIN,<br>ORG_ADMIN,<br>STAFF | VIEW_REPORTS     | Org Scoped          |
| 56 | GET   | /api/v1/reports/:id/pdf                            | BEARER JWT             | SUPER_ADMIN,<br>ORG_ADMIN,<br>STAFF | VIEW_REPORTS     | Report Scoped       |
| 57 | GET   | /api/v1/subscription/plans                         | BEARER JWT             | ORG_ADMIN,<br>SUPER_ADMIN           | NONE             | Platform-wide       |
| 58 | GET   | /api/v1/subscription                               | BEARER JWT             | ORG_ADMIN,<br>SUPER_ADMIN           | NONE             | Own Org             |
| 59 | GET   | /api/v1/subscription/payments                      | BEARER JWT             | ORG_ADMIN,<br>SUPER_ADMIN           | NONE             | Own Org             |
| 60 | POST  | /api/v1/subscription/plan-requests                 | BEARER JWT             | ORG_ADMIN                           | NONE             | Own Org             |
| 61 | POST  | /api/v1/public/cards/resolve                       | PUBLIC                 | PUBLIC                              | NONE             | Public Masked       |
| 62 | POST  | /api/v1/public/sessions/access                     | PUBLIC                 | PUBLIC                              | NONE             | Public Masked       |
| 63 | GET   | /api/v1/public/sessions/:sessionToken              | PUBLIC (Token in Path) | PUBLIC                              | NONE             | Session Scoped      |
| 64 | GET   | /api/v1/public/sessions/:sessionToken/transactions | PUBLIC (Token in Path) | PUBLIC                              | NONE             | Session Scoped      |
| 65 | GET   | /api/v1/public/sessions/:sessionToken/receipts     | PUBLIC (Token in Path) | PUBLIC                              | NONE             | Session Scoped      |

## 9. Complete API Contracts (Request & Response JSON Specifications)

### 1. POST /api/v1/auth/login

**Purpose:** Authenticates platform staff or administrator, issuing signed JWT access token and user session.

**Authentication:** **PUBLIC**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **NONE**

**Organization Scope:** None (Global authentication route)

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

#### Request JSON

```
{
  "email": "staff@example.com",
  "password": "SecurePassword123"
}
```

#### Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJzdGZl8wMDEiLCJyb2xlIjoiaU1RBRkYiLCJleHAiOjE3NzEwMDAwMDB9.sample_token_signature",
    "user": {
      "id": "staff_001",
      "name": "John Doe",
      "email": "staff@example.com",
      "role": "STAFF",
      "organizationId": "org_001",
      "permissions": [
        "CARD_VIEW",
        "CARD_ISSUE",
        "RECHARGE",
        "PURCHASE",
        "REFUND"
      ],
      "assignedBranchIds": [
        "branch_001"
      ]
    }
  }
}
```

#### Applicable Error Responses

##### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Valid email and password are required"
  }
}
```

##### • HTTP 401 **UNAUTHORIZED**:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Invalid email or password"
  }
}
```

**Developer 2 Implementation Notes:** Verify bcrypt password hash. If user account status is INACTIVE, reject immediately with 401 UNAUTHORIZED.



## 2. POST /api/v1/auth/refresh

**Purpose:** Rotates refresh token and issues a fresh 15-minute JWT access token.

**Authentication:** **PUBLIC (Refresh Token in body/cookie)**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **NONE**

**Organization Scope:** None

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

### Request JSON

```
{}
```

### Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJjZdGmZl8wMDEiLCJyb2xlIjoU1RBRkYiLCJleHAiOjE3NzEwMDM2MDB9.new_token_signature"
  }
}
```

### Applicable Error Responses

#### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Refresh token is required"
  }
}
```

#### • HTTP 401 **UNAUTHORIZED**:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Invalid or expired refresh token"
  }
}
```

**Developer 2 Implementation Notes:** Check revocation list in Redis/DB session table before issuing new access token.

## 3. POST /api/v1/auth/logout

**Purpose:** Revokes active session and invalidates the refresh token.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **NONE**

**Organization Scope:** Self identity

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

### Request JSON

```
{}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "loggedOut": true
  }
}
```

## Applicable Error Responses

### • HTTP 401 UNAUTHORIZED:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

**Developer 2 Implementation Notes:** Clear HTTP-only cookies and record token hash in revoked list.

---

## 4. GET /api/v1/auth/me

**Purpose:** Returns authenticated identity profile, organization details, permissions list, and branch assignments.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **NONE**

**Organization Scope:** Scoped to authenticated user's organization

**Branch Scope:** Scoped to user's assigned branches

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "user": {
      "id": "staff_001",
      "name": "John Doe",
      "email": "staff@example.com",
      "role": "STAFF",
      "organizationId": "org_001",
      "permissions": [
        "CARD_VIEW",
        "CARD_ISSUE",
        "RECHARGE",
        "PURCHASE",
        "REFUND"
      ],
      "assignedBranchIds": [
        "branch_001"
      ]
    },
    "organization": {
      "id": "org_001",
      "name": "Acme Cafeteria Group",
      "status": "ACTIVE"
    },
    "permissions": [
      "CARD_VIEW",
      "CARD_ISSUE",
      "RECHARGE",
      "PURCHASE",
      "REFUND"
    ],
    "branches": [
      {
        "id": "branch_001",
        "name": "Main Cafeteria",
        "organizationId": "org_001",
        "status": "ACTIVE"
      }
    ]
  }
}
```

## Applicable Error Responses

- **HTTP 401 UNAUTHORIZED:**

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication token missing or invalid"
  }
}
```

**Developer 2 Implementation Notes:** Query user record by req.user.id including relations to Organization and StaffBranch.

## 5. POST /api/v1/auth/forgot-password

**Purpose:** Initiates password reset workflow. M0 Rule 14.3: Must never leak account existence.

**Authentication:** **PUBLIC**

**Role:** **PUBLIC**

**Permission:** **NONE**

**Organization Scope:** None

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "email": "user@example.com"
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "message": "If the account exists, reset instructions were sent."
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Valid email address is required"
  }
}
```

**Developer 2 Implementation Notes:** Always return HTTP 200 with neutral success message even if email is not found in database.

---

## 6. POST /api/v1/auth/reset-password

**Purpose:** Consumes one-time cryptographic reset token and establishes a new password for account.

**Authentication:** **PUBLIC** (Token in body)

**Role:** **PUBLIC**

**Permission:** **NONE**

**Organization Scope:** None

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "token": "one_time_valid_reset_token_12345",
  "newPassword": "NewSecurePassword456!"
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "reset": true
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid or expired password reset token"
  }
}
```

**Developer 2 Implementation Notes:** Invalidate token immediately upon consumption. Minimum password length 8 characters.

---

## 7. POST /api/v1/auth/change-password

**Purpose:** Allows authenticated user to change their own password after verifying current password.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **NONE**

**Organization Scope:** Self identity

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

### Request JSON

```
{
  "currentPassword": "CurrentPassword123",
  "newPassword": "NewPassword789!"
}
```

### Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "changed": true
  }
}
```

### Applicable Error Responses

#### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Current password is incorrect"
  }
}
```

#### • HTTP 401 **UNAUTHORIZED**:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

**Developer 2 Implementation Notes:** Verify current password using bcrypt before saving newly hashed password.

## 8. GET /api/v1/admin/organizations

**Purpose:** Lists all tenant organizations across the platform with pagination and status.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Platform-wide (All tenants)

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** page (opt, int), limit (opt, int), search (opt, str)

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "org_001",
        "name": "Acme Cafeteria Group",
        "status": "ACTIVE",
        "createdAt": "2026-01-01T00:00:00.000Z",
        "updatedAt": "2026-01-01T00:00:00.000Z"
      },
      {
        "id": "org_002",
        "name": "Zenith Food Services",
        "status": "ACTIVE",
        "createdAt": "2026-01-15T00:00:00.000Z",
        "updatedAt": "2026-01-15T00:00:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 2,
      "totalPages": 1
    }
  }
}
```

## Applicable Error Responses

### • HTTP 401 UNAUTHORIZED:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

### • HTTP 403 FORBIDDEN:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

**Developer 2 Implementation Notes:** Prisma findMany on Organization with pagination envelope.

## 9. POST /api/v1/admin/organizations

**Purpose:** Provisions a new tenant organization and optionally seeds initial Org Admin account.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Platform-wide creation

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "name": "Acme Cafeteria",
  "status": "ACTIVE"
}
```

## Success Response JSON (201 Created)

```
{
  "success": true,
  "data": {
    "id": "org_003",
    "name": "Acme Cafeteria",
    "status": "ACTIVE",
    "createdAt": "2026-02-01T00:00:00.000Z",
    "updatedAt": "2026-02-01T00:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Organization name is required"
  }
}
```

### • HTTP 403 **FORBIDDEN**:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

**Developer 2 Implementation Notes:** Execute organization creation and initial subscription setup in a single \$transaction.

## 10. GET /api/v1/admin/organizations/:id

**Purpose:** Retrieves single organization details including active subscription tier and effective limits.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Target organization ID

**Branch Scope:** None

**Path Parameters:** id: Target Organization ID (e.g. org\_001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "id": "org_001",
    "name": "Acme Cafeteria Group",
    "status": "ACTIVE",
    "createdAt": "2026-01-01T00:00:00.000Z",
    "updatedAt": "2026-01-01T00:00:00.000Z",
    "subscription": {
      "id": "sub_001",
      "organizationId": "org_001",
      "planId": "PLAN_STANDARD",
      "status": "ACTIVE",
      "startDate": "2026-01-01T00:00:00.000Z",
      "endDate": "2026-12-31T23:59:59.000Z",
      "renewalDate": "2026-12-31T23:59:59.000Z",
      "paymentStatus": "SUCCESS"
    },
    "effectiveLimits": {
      "branchLimit": 3,
      "staffLimit": 25,
      "cardLimit": 1000
    }
  }
}
```

## Applicable Error Responses

### • HTTP 403 FORBIDDEN:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

### • HTTP 404 NOT\_FOUND:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Organization 'org_999' not found"
  }
}
```

**Developer 2 Implementation Notes:** Compute effectiveLimits by combining base plan limits with organization overrides.

## 11. PATCH /api/v1/admin/organizations/:id

**Purpose:** Updates organization name or toggles organization status (ACTIVE / INACTIVE).

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Target organization ID

**Branch Scope:** None

**Path Parameters:** id: Target Organization ID (e.g. org\_001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "name": "Acme Premier Dining Group",
  "status": "ACTIVE"
}
```



## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "id": "org_001",
    "name": "Acme Premier Dining Group",
    "status": "ACTIVE",
    "createdAt": "2026-01-01T00:00:00.000Z",
    "updatedAt": "2026-02-15T12:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Organization name cannot be empty"
  }
}
```

### • HTTP 403 **FORBIDDEN**:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

### • HTTP 404 **NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Organization not found"
  }
}
```

**Developer 2 Implementation Notes:** If status is set to INACTIVE, all child staff logins under this org must be rejected.

## 12. GET /api/v1/admin/plans

**Purpose:** Lists all global subscription plan templates. (M0 Rule 21: No transaction\_limit).

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Platform-wide

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```

{
  "success": true,
  "data": {
    "items": [
      {
        "id": "PLAN_BASIC",
        "name": "Basic Plan",
        "status": "ACTIVE",
        "price": 499,
        "currency": "INR",
        "billingInterval": "MONTHLY",
        "branchLimit": 1,
        "staffLimit": 10,
        "cardLimit": 250,
        "inventoryLevel": "Basic",
        "reportsLevel": "Yes",
        "analyticsLevel": "Basic",
        "multiBranchEnabled": false,
        "whiteLabelEnabled": true,
        "supportLevel": "Standard",
        "createdAt": "2026-01-01T00:00:00.000Z",
        "updatedAt": "2026-01-01T00:00:00.000Z"
      },
      {
        "id": "PLAN_STANDARD",
        "name": "Standard Plan",
        "status": "ACTIVE",
        "price": 1499,
        "currency": "INR",
        "billingInterval": "MONTHLY",
        "branchLimit": 3,
        "staffLimit": 25,
        "cardLimit": 1000,
        "inventoryLevel": "Advanced",
        "reportsLevel": "Yes",
        "analyticsLevel": "Standard",
        "multiBranchEnabled": true,
        "whiteLabelEnabled": true,
        "supportLevel": "Priority",
        "createdAt": "2026-01-01T00:00:00.000Z",
        "updatedAt": "2026-01-01T00:00:00.000Z"
      },
      {
        "id": "PLAN_PRO",
        "name": "Pro Plan",
        "status": "ACTIVE",
        "price": 2999,
        "currency": "INR",
        "billingInterval": "MONTHLY",
        "branchLimit": 10,
        "staffLimit": 75,
        "cardLimit": 5000,
        "inventoryLevel": "Advanced",
        "reportsLevel": "Yes",
        "analyticsLevel": "Advanced",
        "multiBranchEnabled": true,
        "whiteLabelEnabled": true,
        "supportLevel": "Priority",
        "createdAt": "2026-01-01T00:00:00.000Z",
        "updatedAt": "2026-01-01T00:00:00.000Z"
      },
      {
        "id": "PLAN_ENTERPRISE",
        "name": "Enterprise Plan",
        "status": "ACTIVE",
        "price": 9999,
        "currency": "INR",
        "billingInterval": "YEARLY",
        "branchLimit": 999,
        "staffLimit": 999,
        "cardLimit": 99999,
        "inventoryLevel": "Advanced",
        "reportsLevel": "Yes",
        "analyticsLevel": "Advanced",
        "multiBranchEnabled": true,
        "whiteLabelEnabled": true,
        "supportLevel": "Dedicated",
        "createdAt": "2026-01-01T00:00:00.000Z",
        "updatedAt": "2026-01-01T00:00:00.000Z"
      }
    ]
  }
}

```

```

      "updatedAt": "2026-01-01T00:00:00.000Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 4,
    "totalPages": 1
  }
}

```

## Applicable Error Responses

### • HTTP 401 UNAUTHORIZED:

```

{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}

```

**Developer 2 Implementation Notes:** Plans do not contain transaction\_limit. Return branchLimit, staffLimit, cardLimit.

## 13. POST /api/v1/admin/plans

**Purpose:** Creates a new global subscription plan template.

**Authentication:** [BEARER JWT](#)

**Role:** [SUPER\\_ADMIN](#)

**Permission:** [NONE](#)

**Organization Scope:** Platform-wide

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** [application/json](#)

### Request JSON

```

{
  "name": "Custom Corporate",
  "price": 3999,
  "currency": "INR",
  "billingInterval": "MONTHLY",
  "branchLimit": 15,
  "staffLimit": 100,
  "cardLimit": 8000,
  "inventoryLevel": "Advanced",
  "reportsLevel": "Yes",
  "analyticsLevel": "Advanced",
  "multiBranchEnabled": true,
  "whiteLabelEnabled": true,
  "supportLevel": "Priority",
  "status": "ACTIVE"
}

```

## Success Response JSON (201 Created)

```
{
  "success": true,
  "data": {
    "id": "plan_custom_001",
    "name": "Custom Corporate",
    "status": "ACTIVE",
    "price": 3999,
    "currency": "INR",
    "billingInterval": "MONTHLY",
    "branchLimit": 15,
    "staffLimit": 100,
    "cardLimit": 8000,
    "inventoryLevel": "Advanced",
    "reportsLevel": "Yes",
    "analyticsLevel": "Advanced",
    "multiBranchEnabled": true,
    "whiteLabelEnabled": true,
    "supportLevel": "Priority",
    "createdAt": "2026-02-15T12:00:00.000Z",
    "updatedAt": "2026-02-15T12:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Valid plan name and non-negative price required"
  }
}
```

### • HTTP 403 **FORBIDDEN**:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

**Developer 2 Implementation Notes:** Generate immutable uppercase ID slug if ID not provided.

## 14. GET /api/v1/admin/plans/:id

**Purpose:** Retrieves single subscription plan template definition.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Platform-wide

**Branch Scope:** None

**Path Parameters:** id: Target Plan ID (e.g. PLAN\_PRO)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "id": "PLAN_PRO",
    "name": "Pro Plan",
    "status": "ACTIVE",
    "price": 2999,
    "currency": "INR",
    "billingInterval": "MONTHLY",
    "branchLimit": 10,
    "staffLimit": 75,
    "cardLimit": 5000,
    "inventoryLevel": "Advanced",
    "reportsLevel": "Yes",
    "analyticsLevel": "Advanced",
    "multiBranchEnabled": true,
    "whiteLabelEnabled": true,
    "supportLevel": "Priority",
    "createdAt": "2026-01-01T00:00:00.000Z",
    "updatedAt": "2026-01-01T00:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 404 NOT\_FOUND:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Plan not found"
  }
}
```

**Developer 2 Implementation Notes:** Query `prisma.plan.findUnique({ where: { id } })`.

## 15. PATCH /api/v1/admin/plans/:id

**Purpose:** Updates an existing global subscription plan template's pricing, limits, or features.

**Authentication:** [BEARER JWT](#)

**Role:** [SUPER\\_ADMIN](#)

**Permission:** [NONE](#)

**Organization Scope:** Platform-wide

**Branch Scope:** None

**Path Parameters:** id: Target Plan ID (e.g. PLAN\_PRO)

**Query Parameters:** None

**Request Content-Type:** [application/json](#)

## Request JSON

```
{
  "name": "Pro Plus Plan",
  "price": 3299,
  "cardLimit": 6000
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "id": "PLAN_PRO",
    "name": "Pro Plus Plan",
    "status": "ACTIVE",
    "price": 3299,
    "currency": "INR",
    "billingInterval": "MONTHLY",
    "branchLimit": 10,
    "staffLimit": 75,
    "cardLimit": 6000,
    "inventoryLevel": "Advanced",
    "reportsLevel": "Yes",
    "analyticsLevel": "Advanced",
    "multiBranchEnabled": true,
    "whiteLabelEnabled": true,
    "supportLevel": "Priority",
    "createdAt": "2026-01-01T00:00:00.000Z",
    "updatedAt": "2026-02-15T12:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Plan price must be non-negative"
  }
}
```

### • HTTP 403 **FORBIDDEN**:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

### • HTTP 404 **NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Plan not found"
  }
}
```

**Developer 2 Implementation Notes:** Plan ID remains immutable.

## 16. GET /api/v1/admin/organizations/:id/subscription

**Purpose:** Retrieves an organization's active subscription, base plan metadata, and custom limit overrides.

**Authentication:** **BEARER** **JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Target organization ID

**Branch Scope:** None

**Path Parameters:** id: Organization ID (e.g. org\_001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "subscription": {
      "id": "sub_001",
      "organizationId": "org_001",
      "planId": "PLAN_STANDARD",
      "status": "ACTIVE",
      "startDate": "2026-01-01T00:00:00.000Z",
      "endDate": "2026-12-31T23:59:59.000Z",
      "renewalDate": "2026-12-31T23:59:59.000Z",
      "paymentStatus": "SUCCESS"
    },
    "plan": {
      "id": "PLAN_STANDARD",
      "name": "Standard Plan",
      "branchLimit": 3,
      "staffLimit": 25,
      "cardLimit": 1000
    },
    "effectiveLimits": {
      "branchLimit": 4,
      "staffLimit": 30,
      "cardLimit": 1500
    },
    "overrides": {
      "branchLimit": 4,
      "staffLimit": 30,
      "cardLimit": 1500
    }
  }
}
```

## Applicable Error Responses

### • HTTP 403 FORBIDDEN:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

### • HTTP 404 NOT\_FOUND:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Subscription for organization not found"
  }
}
```

**Developer 2 Implementation Notes:** Compute effectiveLimits = override ?? plan.limit for each resource.

## 17. PATCH /api/v1/admin/organizations/:id/subscription

**Purpose:** Directly assigns a new plan tier or updates subscription status for an organization.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Target organization ID

**Branch Scope:** None

**Path Parameters:** id: Organization ID (e.g. org\_001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "planId": "PLAN_PRO",
  "status": "ACTIVE",
  "startDate": "2026-02-01T00:00:00.000Z",
  "renewalDate": "2027-02-01T00:00:00.000Z"
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "subscription": {
      "id": "sub_001",
      "organizationId": "org_001",
      "planId": "PLAN_PRO",
      "status": "ACTIVE",
      "startDate": "2026-02-01T00:00:00.000Z",
      "endDate": "2027-02-01T00:00:00.000Z",
      "renewalDate": "2027-02-01T00:00:00.000Z",
      "paymentStatus": "SUCCESS"
    },
    "effectiveLimits": {
      "branchLimit": 10,
      "staffLimit": 75,
      "cardLimit": 5000
    }
  }
}
```

## Applicable Error Responses

### • HTTP 403 FORBIDDEN:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

### • HTTP 409 DOWNGRADE\_LIMIT\_EXCEEDED:

```
{
  "success": false,
  "error": {
    "code": "DOWNGRADE_LIMIT_EXCEEDED",
    "message": "Cannot apply plan: Current active branches exceed target limit"
  }
}
```

**Developer 2 Implementation Notes:** Verify current usage counts do not exceed target plan limits before updating.

## 18. GET /api/v1/admin/organizations/:id/limit-overrides

**Purpose:** Retrieves custom organization limit overrides set by Super Admin.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Target organization ID

**Branch Scope:** None

**Path Parameters:** id: Organization ID (e.g. org\_001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*



## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "overrides": {
      "branchLimit": 5,
      "staffLimit": 40,
      "cardLimit": 2500
    },
    "effectiveLimits": {
      "branchLimit": 5,
      "staffLimit": 40,
      "cardLimit": 2500
    }
  }
}
```

## Applicable Error Responses

### • HTTP 403 FORBIDDEN:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

### • HTTP 404 NOT\_FOUND:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Organization not found"
  }
}
```

**Developer 2 Implementation Notes:** Overrides are stored as JSON column on the Subscription model.

## 19. PATCH /api/v1/admin/organizations/:id/limit-overrides

**Purpose:** Sets custom quota overrides for branches, staff, or cards for an organization.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Target organization ID

**Branch Scope:** None

**Path Parameters:** id: Organization ID (e.g. org\_001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "branchLimit": 4,
  "staffLimit": 40,
  "cardLimit": 2500
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "overrides": {
      "branchLimit": 4,
      "staffLimit": 40,
      "cardLimit": 2500
    },
    "effectiveLimits": {
      "branchLimit": 4,
      "staffLimit": 40,
      "cardLimit": 2500
    }
  }
}
```

## Applicable Error Responses

### • HTTP 403 FORBIDDEN:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

### • HTTP 409 PLAN\_LIMIT\_REACHED:

```
{
  "success": false,
  "error": {
    "code": "PLAN_LIMIT_REACHED",
    "message": "Cannot reduce branchLimit below current count of active branches"
  }
}
```

**Developer 2 Implementation Notes:** Passing null for an override key resets that specific limit to base plan default.

## 20. GET /api/v1/admin/subscriptions

**Purpose:** Lists all organization subscription records across the platform.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Platform-wide

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** page (opt, int), limit (opt, int)

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "sub_001",
        "organizationId": "org_001",
        "planId": "PLAN_STANDARD",
        "status": "ACTIVE",
        "startDate": "2026-01-01T00:00:00.000Z",
        "endDate": "2026-12-31T23:59:59.000Z",
        "renewalDate": "2026-12-31T23:59:59.000Z",
        "paymentStatus": "SUCCESS",
        "createdAt": "2026-01-01T00:00:00.000Z",
        "updatedAt": "2026-01-01T00:00:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 1,
      "totalPages": 1
    }
  }
}
```

## Applicable Error Responses

### • HTTP 401 UNAUTHORIZED:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

### • HTTP 403 FORBIDDEN:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

**Developer 2 Implementation Notes:** Query `prisma.subscription.findMany()` with pagination.

## 21. GET /api/v1/admin/subscriptions/:id

**Purpose:** Retrieves single subscription audit details including direct payment transactions.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Platform-wide

**Branch Scope:** None

**Path Parameters:** id: Subscription ID (e.g. sub\_001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "id": "sub_001",
    "organizationId": "org_001",
    "planId": "PLAN_STANDARD",
    "status": "ACTIVE",
    "startDate": "2026-01-01T00:00:00.000Z",
    "endDate": "2026-12-31T23:59:59.000Z",
    "renewalDate": "2026-12-31T23:59:59.000Z",
    "paymentStatus": "SUCCESS",
    "overrides": {
      "branchLimit": 4,
      "staffLimit": 30,
      "cardLimit": 1500
    },
    "payments": [
      {
        "id": "sub_pay_001",
        "subscriptionId": "sub_001",
        "amount": 1499,
        "currency": "INR",
        "status": "SUCCESS",
        "paymentMethod": "DIRECT_BANK_TRANSFER",
        "externalReference": "NEFT_982341203",
        "createdAt": "2026-01-01T00:00:00.000Z"
      }
    ],
    "createdAt": "2026-01-01T00:00:00.000Z",
    "updatedAt": "2026-01-01T00:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 403 FORBIDDEN:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

### • HTTP 404 NOT\_FOUND:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Subscription not found"
  }
}
```

**Developer 2 Implementation Notes:** Include payments relation in Prisma query.

## 22. GET /api/v1/admin/subscription-payments

**Purpose:** Lists all direct subscription payment entries across all organizations.

**Authentication:** **BEARER** **JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Platform-wide

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** page (opt, int), limit (opt, int)

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "sub_pay_001",
        "subscriptionId": "sub_001",
        "organizationId": "org_001",
        "amount": 1499,
        "currency": "INR",
        "status": "SUCCESS",
        "paymentMethod": "DIRECT_BANK_TRANSFER",
        "paymentReference": "DIRECT_NEFT_982341203",
        "externalReference": "NEFT_982341203",
        "createdAt": "2026-01-01T00:00:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 1,
      "totalPages": 1
    }
  }
}
```

## Applicable Error Responses

### • HTTP 403 FORBIDDEN:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

**Developer 2 Implementation Notes:** M0 V10 Rule: Subscriptions use direct bank transfers and offline invoices.

## 23. GET /api/v1/admin/plan-change-requests

**Purpose:** Lists all pending and resolved plan change requests submitted by tenant org admins.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Platform-wide

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** status (opt, str: PENDING, APPROVED, REJECTED), page, limit

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "PCR001",
        "organizationId": "org_001",
        "organizationName": "Acme Cafeteria Group",
        "requestedByUserId": "staff_admin_001",
        "currentPlanId": "PLAN_STANDARD",
        "currentPlanName": "Standard Plan",
        "requestedPlanId": "PLAN_PRO",
        "requestedPlanName": "Pro Plan",
        "requestType": "UPGRADE",
        "message": "Need higher staff and card limits for new campus branch.",
        "status": "PENDING",
        "adminNote": null,
        "createdAt": "2026-02-10T10:00:00.000Z",
        "updatedAt": "2026-02-10T10:00:00.000Z",
        "resolvedAt": null
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 1,
      "totalPages": 1
    }
  }
}
```

## Applicable Error Responses

### • HTTP 403 FORBIDDEN:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

**Developer 2 Implementation Notes:** Order by status = PENDING first, then createdAt DESC.

## 24. GET /api/v1/admin/plan-change-requests/:id

**Purpose:** Retrieves details of a specific plan change request.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Platform-wide

**Branch Scope:** None

**Path Parameters:** id: Plan Change Request ID (e.g. PCR001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "id": "PCR001",
    "organizationId": "org_001",
    "organizationName": "Acme Cafeteria Group",
    "requestedByUserId": "staff_admin_001",
    "currentPlanId": "PLAN_STANDARD",
    "currentPlanName": "Standard Plan",
    "requestedPlanId": "PLAN_PRO",
    "requestedPlanName": "Pro Plan",
    "requestType": "UPGRADE",
    "message": "Need higher staff and card limits for new campus branch.",
    "status": "PENDING",
    "adminNote": null,
    "createdAt": "2026-02-10T10:00:00.000Z",
    "updatedAt": "2026-02-10T10:00:00.000Z",
    "resolvedAt": null
  }
}
```

## Applicable Error Responses

### • HTTP 403 FORBIDDEN:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

### • HTTP 404 NOT\_FOUND:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Plan change request not found"
  }
}
```

**Developer 2 Implementation Notes:** Query `prisma.planChangeRequest.findUnique({ where: { id } })`.

## 25. PATCH /api/v1/admin/plan-change-requests/:id

**Purpose:** Approves or rejects a plan change request, automatically transitioning the org subscription on approval.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Platform-wide

**Branch Scope:** None

**Path Parameters:** id: Plan Change Request ID (e.g. PCR001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "status": "APPROVED",
  "adminNote": "Approved after contract review."
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "id": "PCR001",
    "organizationId": "org_001",
    "requestedByUserId": "staff_admin_001",
    "currentPlanId": "PLAN_STANDARD",
    "requestedPlanId": "PLAN_PRO",
    "requestType": "UPGRADE",
    "message": "Need higher staff and card limits for new campus branch.",
    "status": "APPROVED",
    "adminNote": "Approved after contract review.",
    "createdAt": "2026-02-10T10:00:00.000Z",
    "updatedAt": "2026-02-15T12:00:00.000Z",
    "resolvedAt": "2026-02-15T12:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Status must be APPROVED or REJECTED"
  }
}
```

### • HTTP 403 **FORBIDDEN**:

```
{
  "success": false,
  "error": {
    "code": "FORBIDDEN",
    "message": "Super Admin access required"
  }
}
```

### • HTTP 404 **NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Plan change request not found"
  }
}
```

**Developer 2 Implementation Notes:** Execute request update and subscription.planId update in a single Prisma transaction.

## 26. GET /api/v1/organization

**Purpose:** Returns current tenant organization profile, active subscription, and effective limits for logged-in Org Admin.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*



## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "organization": {
      "id": "org_001",
      "name": "Acme Cafeteria Group",
      "status": "ACTIVE",
      "createdAt": "2026-01-01T00:00:00.000Z",
      "updatedAt": "2026-01-01T00:00:00.000Z"
    },
    "subscription": {
      "id": "sub_001",
      "organizationId": "org_001",
      "planId": "PLAN_STANDARD",
      "status": "ACTIVE",
      "startDate": "2026-01-01T00:00:00.000Z",
      "endDate": "2026-12-31T23:59:59.000Z",
      "renewalDate": "2026-12-31T23:59:59.000Z",
      "paymentStatus": "SUCCESS"
    },
    "effectiveLimits": {
      "branchLimit": 3,
      "staffLimit": 25,
      "cardLimit": 1000
    }
  }
}
```

## Applicable Error Responses

### • HTTP 401 UNAUTHORIZED:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

### • HTTP 404 NOT\_FOUND:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Organization not found"
  }
}
```

**Developer 2 Implementation Notes:** Filter strictly by organizationId = req.user.organizationId.

## 27. PATCH /api/v1/organization

**Purpose:** Updates current organization display name.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "name": "Acme Cafeteria Network"
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "id": "org_001",
    "name": "Acme Cafeteria Network",
    "status": "ACTIVE",
    "createdAt": "2026-01-01T00:00:00.000Z",
    "updatedAt": "2026-02-15T12:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Organization name is required"
  }
}
```

### • HTTP 401 **UNAUTHORIZED**:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

**Developer 2 Implementation Notes:** Update name in `prisma.organization.update({ where: { id: req.user.organizationId } })`.

## 28. GET /api/v1/branches

**Purpose:** Lists cafeteria branches accessible to authenticated user within their organization.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **BRANCH\_VIEW**

**Organization Scope:** Strictly scoped to `req.user.organizationId`

**Branch Scope:** STAFF is filtered to `assignedBranchIds`

**Path Parameters:** None

**Query Parameters:** page, limit, search

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "branch_001",
        "organizationId": "org_001",
        "name": "Main Cafeteria",
        "status": "ACTIVE",
        "createdAt": "2026-01-01T00:00:00.000Z",
        "updatedAt": "2026-01-01T00:00:00.000Z"
      },
      {
        "id": "branch_002",
        "organizationId": "org_001",
        "name": "Executive Lounge",
        "status": "ACTIVE",
        "createdAt": "2026-01-05T00:00:00.000Z",
        "updatedAt": "2026-01-05T00:00:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 2,
      "totalPages": 1
    }
  }
}
```

## Applicable Error Responses

### • HTTP 401 UNAUTHORIZED:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

### • HTTP 403 PERMISSION\_DENIED:

```
{
  "success": false,
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "Missing required permission: BRANCH_VIEW"
  }
}
```

**Developer 2 Implementation Notes:** Filter by organizationId. If role === STAFF, intersect with assignedBranchIds.

## 29. POST /api/v1/branches

**Purpose:** Creates a new cafeteria branch under current tenant organization, enforcing effective branch limits.

**Authentication:** BEARER JWT

**Role:** ORG\_ADMIN, SUPER\_ADMIN

**Permission:** BRANCH\_MANAGE

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** application/json

## Request JSON

```
{
  "name": "North Campus Kiosk"
}
```

## Success Response JSON (201 Created)

```
{
  "success": true,
  "data": {
    "id": "branch_003",
    "organizationId": "org_001",
    "name": "North Campus Kiosk",
    "status": "ACTIVE",
    "createdAt": "2026-02-15T12:00:00.000Z",
    "updatedAt": "2026-02-15T12:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Branch name is required"
  }
}
```

### • HTTP 403 **PERMISSION\_DENIED**:

```
{
  "success": false,
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "Missing required permission: BRANCH_MANAGE"
  }
}
```

### • HTTP 409 **PLAN\_LIMIT\_REACHED**:

```
{
  "success": false,
  "error": {
    "code": "PLAN_LIMIT_REACHED",
    "message": "Plan limit reached: Maximum allowed branches reached"
  }
}
```

**Developer 2 Implementation Notes:** Verify current branch count < effectiveBranchLimit before insertion.

## 30. GET /api/v1/branches/:id

**Purpose:** Retrieves single branch details.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **BRANCH\_VIEW**

**Organization Scope:** Strictly checked: branch.organizationId === req.user.organizationId

**Branch Scope:** Staff must be assigned to :id

**Path Parameters:** id: Target Branch ID (e.g. branch\_001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "id": "branch_001",
    "organizationId": "org_001",
    "name": "Main Cafeteria",
    "status": "ACTIVE",
    "createdAt": "2026-01-01T00:00:00.000Z",
    "updatedAt": "2026-01-01T00:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 403 **BRANCH\_ACCESS\_DENIED**:

```
{
  "success": false,
  "error": {
    "code": "BRANCH_ACCESS_DENIED",
    "message": "You do not have access to this branch"
  }
}
```

### • HTTP 404 **NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Branch not found"
  }
}
```

**Developer 2 Implementation Notes:** Enforce both organization boundary and staff assignment check.

## 31. PATCH /api/v1/branches/:id

**Purpose:** Updates branch name or status.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, SUPER\_ADMIN**

**Permission:** **BRANCH\_MANAGE**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Target branch must belong to organization

**Path Parameters:** id: Target Branch ID (e.g. branch\_001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "name": "Main Central Cafeteria",
  "status": "ACTIVE"
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "id": "branch_001",
    "organizationId": "org_001",
    "name": "Main Central Cafeteria",
    "status": "ACTIVE",
    "createdAt": "2026-01-01T00:00:00.000Z",
    "updatedAt": "2026-02-15T12:00:00.000Z"
  }
}
```

## Applicable Error Responses

- **HTTP 400 VALIDATION\_ERROR:**

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Branch name cannot be empty"
  }
}
```

- **HTTP 403 BRANCH\_ACCESS\_DENIED:**

```
{
  "success": false,
  "error": {
    "code": "BRANCH_ACCESS_DENIED",
    "message": "You do not have permission to manage this branch"
  }
}
```

- **HTTP 404 NOT\_FOUND:**

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Branch not found"
  }
}
```

**Developer 2 Implementation Notes:** Verify organizationId matches before applying update.

## 32. GET /api/v1/staff

**Purpose:** Lists all staff accounts for organization with assigned branches and permissions array.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, SUPER\_ADMIN**

**Permission:** **STAFF\_VIEW**

**Organization Scope:** Strictly scoped to req.user.organizationId

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** page, limit, search

**Request Content-Type:** **application/json**

### Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "staff_001",
        "organizationId": "org_001",
        "name": "John Doe",
        "email": "john@example.com",
        "status": "ACTIVE",
        "permissions": [
          "CARD_VIEW",
          "CARD_ISSUE",
          "RECHARGE",
          "PURCHASE",
          "REFUND"
        ],
        "assignedBranchIds": [
          "branch_001"
        ],
        "createdAt": "2026-01-01T00:00:00.000Z",
        "updatedAt": "2026-01-01T00:00:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 1,
      "totalPages": 1
    }
  }
}
```

## Applicable Error Responses

### • HTTP 401 UNAUTHORIZED:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

### • HTTP 403 PERMISSION\_DENIED:

```
{
  "success": false,
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "Missing required permission: STAFF_VIEW"
  }
}
```

**Developer 2 Implementation Notes:** Filter by organizationId = req.user.organizationId.

## 33. POST /api/v1/staff

**Purpose:** Creates a new staff member account, establishes initial password hash, and assigns branches/permissions.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, SUPER\_ADMIN**

**Permission:** **STAFF\_MANAGE**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Assigned branches must belong to organization

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "name": "Sarah Connor",
  "email": "sarah@example.com",
  "permissions": [
    "CARD_VIEW",
    "CARD_ISSUE",
    "RECHARGE",
    "PURCHASE"
  ],
  "branchIds": [
    "branch_001"
  ]
}
```

## Success Response JSON (201 Created)

```
{
  "success": true,
  "data": {
    "id": "staff_002",
    "organizationId": "org_001",
    "name": "Sarah Connor",
    "email": "sarah@example.com",
    "status": "ACTIVE",
    "permissions": [
      "CARD_VIEW",
      "CARD_ISSUE",
      "RECHARGE",
      "PURCHASE"
    ],
    "assignedBranchIds": [
      "branch_001"
    ],
    "createdAt": "2026-02-15T12:00:00.000Z",
    "updatedAt": "2026-02-15T12:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Valid staff name and email are required"
  }
}
```

### • HTTP 403 **PERMISSION\_DENIED**:

```
{
  "success": false,
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "Missing required permission: STAFF_MANAGE"
  }
}
```

### • HTTP 409 **PLAN\_LIMIT\_REACHED**:

```
{
  "success": false,
  "error": {
    "code": "PLAN_LIMIT_REACHED",
    "message": "Staff limit reached for your active subscription"
  }
}
```

**Developer 2 Implementation Notes:** Verify `currentStaffCount < effectiveStaffLimit`. Hash default password with bcrypt.

## 34. PUT /api/v1/staff/:id/branches

**Purpose:** Replaces the entire set of branch assignments for a staff member.

**Authentication:** **BEARER JWT**



**Role:** **ORG\_ADMIN, SUPER\_ADMIN**

**Permission:** **STAFF\_MANAGE**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** All branches must belong to org

**Path Parameters:** id: Staff ID (e.g. staff\_001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "branchIds": [
    "branch_001",
    "branch_002"
  ]
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "staffId": "staff_001",
    "assignedBranchIds": [
      "branch_001",
      "branch_002"
    ]
  }
}
```

## Applicable Error Responses

### • HTTP 403 **ORGANIZATION\_ACCESS\_DENIED**:

```
{
  "success": false,
  "error": {
    "code": "ORGANIZATION_ACCESS_DENIED",
    "message": "One or more branches do not belong to your organization"
  }
}
```

### • HTTP 404 **NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Staff member not found"
  }
}
```

**Developer 2 Implementation Notes:** Prisma deleteMany on StaffBranch and createMany in transaction.

## 35. PUT /api/v1/staff/:id/permissions

**Purpose:** Replaces the set of operational permissions granted to a staff member.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, SUPER\_ADMIN**

**Permission:** **STAFF\_MANAGE**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** None

**Path Parameters:** id: Staff ID (e.g. staff\_001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "permissions": [
    "CARD_VIEW",
    "CARD_ISSUE",
    "RECHARGE",
    "PURCHASE"
  ]
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "staffId": "staff_001",
    "permissions": [
      "CARD_VIEW",
      "CARD_ISSUE",
      "RECHARGE",
      "PURCHASE"
    ]
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid permission code provided"
  }
}
```

### • HTTP 404 **NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Staff member not found"
  }
}
```

**Developer 2 Implementation Notes:** Validate that all provided permission strings belong to the 20 frozen M0 codes.

## 36. GET /api/v1/permissions

**Purpose:** Returns the authoritative system list of exactly 20 frozen M0 permission codes.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **NONE**

**Organization Scope:** None (Global static catalog)

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": [
    { "code": "CARD_VIEW", "name": "View Cards", "area": "Cards" },
    { "code": "CARD_ISSUE", "name": "Issue Cards", "area": "Cards" },
    { "code": "CARD_RETURN", "name": "Return Cards", "area": "Cards" },
    { "code": "CARD_BLOCK", "name": "Block Cards", "area": "Cards" },
    { "code": "CARD_UNBLOCK", "name": "Unblock Cards", "area": "Cards" },
    { "code": "RECHARGE", "name": "Recharge Sessions", "area": "Sessions / Payments" },
    { "code": "PURCHASE", "name": "Process Purchases", "area": "Sessions / Payments" },
    { "code": "REFUND", "name": "Process Refunds", "area": "Sessions / Payments" },
    { "code": "SESSION_VIEW", "name": "View Sessions", "area": "Sessions / Payments" },
    { "code": "PRODUCT_VIEW", "name": "View Products", "area": "Products / Inventory" },
    { "code": "PRODUCT_MANAGE", "name": "Manage Products", "area": "Products / Inventory" },
    { "code": "INVENTORY_VIEW", "name": "View Inventory", "area": "Products / Inventory" },
    { "code": "INVENTORY_MANAGE", "name": "Manage Inventory", "area": "Products / Inventory" },
    { "code": "INVENTORY_IMPORT", "name": "Import Inventory CSV", "area": "Products / Inventory" },
    { "code": "VIEW_ANALYTICS", "name": "View Analytics", "area": "Analytics / Reports" },
    { "code": "VIEW_REPORTS", "name": "View Reports", "area": "Analytics / Reports" },
    { "code": "STAFF_VIEW", "name": "View Staff", "area": "Staff" },
    { "code": "STAFF_MANAGE", "name": "Manage Staff", "area": "Staff" },
    { "code": "BRANCH_VIEW", "name": "View Branches", "area": "Branch" },
    { "code": "BRANCH_MANAGE", "name": "Manage Branches", "area": "Branch" }
  ]
}
```

## Applicable Error Responses

### • HTTP 401 UNAUTHORIZED:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

**Developer 2 Implementation Notes:** Return constant enum catalog array of exactly 20 items.

## 37. GET /api/v1/cards

**Purpose:** Lists cards belonging to organization with status filter (AVAILABLE, ACTIVE, BLOCKED).

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **CARD\_VIEW**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Optionally filtered by branchId

**Path Parameters:** None

**Query Parameters:** status (opt), branchId (opt), search (opt), page, limit

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "CARD001",
        "organizationId": "org_001",
        "currentBranchId": "branch_001",
        "physicalCardNumber": "MC-001",
        "qrToken": "qr_opaque_hash_card_001",
        "status": "AVAILABLE",
        "createdAt": "2026-01-01T00:00:00.000Z",
        "updatedAt": "2026-01-01T00:00:00.000Z"
      },
      {
        "id": "CARD002",
        "organizationId": "org_001",
        "currentBranchId": "branch_001",
        "physicalCardNumber": "MC-002",
        "qrToken": "qr_opaque_hash_card_002",
        "status": "ACTIVE",
        "createdAt": "2026-01-01T00:00:00.000Z",
        "updatedAt": "2026-01-10T10:00:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 2,
      "totalPages": 1
    }
  }
}
```

## Applicable Error Responses

### • HTTP 401 UNAUTHORIZED:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

### • HTTP 403 PERMISSION\_DENIED:

```
{
  "success": false,
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "Missing required permission: CARD_VIEW"
  }
}
```

**Developer 2 Implementation Notes:** Filter by organizationId = req.user.organizationId.

## 38. POST /api/v1/cards

**Purpose:** Creates a physical card record in AVAILABLE state with unique barcode and cryptographic QR token.

**Authentication:** BEARER JWT

**Role:** ORG\_ADMIN, STAFF

**Permission:** CARD\_ISSUE

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Scoped to branchId

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** application/json

## Request JSON

```
{
  "physicalCardNumber": "MC-003",
  "branchId": "branch_001"
}
```

## Success Response JSON (201 Created)

```
{
  "success": true,
  "data": {
    "id": "CARD003",
    "organizationId": "org_001",
    "currentBranchId": "branch_001",
    "physicalCardNumber": "MC-003",
    "qrToken": "qr_opaque_hash_card_003",
    "status": "AVAILABLE",
    "createdAt": "2026-02-15T12:00:00.000Z",
    "updatedAt": "2026-02-15T12:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Physical card number is required"
  }
}
```

### • HTTP 409 **PLAN\_LIMIT\_REACHED**:

```
{
  "success": false,
  "error": {
    "code": "PLAN_LIMIT_REACHED",
    "message": "Card limit reached for your active subscription"
  }
}
```

**Developer 2 Implementation Notes:** Initial card status is ALWAYS AVAILABLE. Generate secure non-guessable qrToken.

## 39. POST /api/v1/cards/resolve

**Purpose:** Resolves a card by QR token or physical barcode during POS workflow, returning status and active session ID.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, STAFF**

**Permission:** **CARD\_VIEW**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Scoped to assigned branch

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "qrToken": "qr_opaque_hash_card_001"
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "cardId": "CARD001",
    "physicalCardNumber": "MC-001",
    "status": "AVAILABLE",
    "branchId": "branch_001",
    "activeSessionId": null,
    "activeBalance": 0
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "QR token or card barcode is required"
  }
}
```

### • HTTP 404 **CARD\_NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "CARD_NOT_FOUND",
    "message": "Invalid or unrecognized QR token"
  }
}
```

**Developer 2 Implementation Notes:** If card has active session, return activeSessionId and activeBalance.

## 40. POST /api/v1/cards/:id/block

**Purpose:** Transitions a card into BLOCKED state, preventing any further recharge, purchase, or issue.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, STAFF**

**Permission:** **CARD\_BLOCK**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** None

**Path Parameters:** id: Target Card ID (e.g. CARD001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "reason": "Physical card lost by customer"
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "id": "CARD001",
    "status": "BLOCKED",
    "updatedAt": "2026-02-15T12:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 403 **PERMISSION\_DENIED**:

```
{
  "success": false,
```

```

    "error": {
      "code": "PERMISSION_DENIED",
      "message": "Missing required permission: CARD_BLOCK"
    }
  }
}

```

• HTTP 404 **CARD\_NOT\_FOUND**:

```

{
  "success": false,
  "error": {
    "code": "CARD_NOT_FOUND",
    "message": "Card not found"
  }
}

```

**Developer 2 Implementation Notes:** Record audit log entry for block action with staff ID and reason.

## 41. POST /api/v1/cards/:id/unblock

**Purpose:** Restores a BLOCKED card back to AVAILABLE state.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, STAFF**

**Permission:** **CARD\_UNBLOCK**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** None

**Path Parameters:** id: Target Card ID (e.g. CARD001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

### Request JSON

```

{}

```

### Success Response JSON (200 OK)

```

{
  "success": true,
  "data": {
    "id": "CARD001",
    "status": "AVAILABLE",
    "updatedAt": "2026-02-15T12:05:00.000Z"
  }
}

```

### Applicable Error Responses

• HTTP 403 **PERMISSION\_DENIED**:

```

{
  "success": false,
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "Missing required permission: CARD_UNBLOCK"
  }
}

```

• HTTP 404 **CARD\_NOT\_FOUND**:

```

{
  "success": false,
  "error": {
    "code": "CARD_NOT_FOUND",
    "message": "Card not found"
  }
}

```

**Developer 2 Implementation Notes:** Reset status to AVAILABLE.

## 42. POST /api/v1/card-sessions

**Purpose:** Starts an ACTIVE session on an AVAILABLE card. Transitions card status to ACTIVE.

**Authentication:** **BEARER JWT**

**Role:** **STAFF, ORG\_ADMIN**

**Permission:** **CARD\_ISSUE**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Scoped to branchId

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "cardId": "CARD001",
  "branchId": "branch_001"
}
```

## Success Response JSON (201 Created)

```
{
  "success": true,
  "data": {
    "id": "SESSION001",
    "cardId": "CARD001",
    "branchId": "branch_001",
    "balance": 0,
    "status": "ACTIVE",
    "startedAt": "2026-02-15T12:00:00.000Z",
    "settledAt": null,
    "createdAt": "2026-02-15T12:00:00.000Z",
    "updatedAt": "2026-02-15T12:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 404 **CARD\_NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "CARD_NOT_FOUND",
    "message": "Card not found"
  }
}
```

### • HTTP 409 **CARD\_NOT\_AVAILABLE**:

```
{
  "success": false,
  "error": {
    "code": "CARD_NOT_AVAILABLE",
    "message": "Card is currently ACTIVE and cannot open another session"
  }
}
```

**Developer 2 Implementation Notes:** Check card.status === AVAILABLE. In a transaction, create CardSession with balance = 0 and update card.status = ACTIVE.

## 43. POST /api/v1/card-sessions/:id/recharge

**Purpose:** Adds funds to an active card session using CASH or verified UPI payment method.

**Authentication:** **BEARER JWT**

**Role:** **STAFF, ORG\_ADMIN**

**Permission:** **RECHARGE**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Scoped to session branch

**Path Parameters:** id: Card Session ID (e.g. SESSION001)

**Query Parameters:** None



**Request Content-Type:** `application/json`

### Request JSON

```
{
  "amount": 500,
  "paymentMethod": "CASH",
  "externalReference": "MANUAL_RECEIPT_091"
}
```

### Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "sessionId": "SESSION001",
    "paymentId": "PAYMENT001",
    "amount": 500,
    "balance": 500,
    "status": "ACTIVE"
  }
}
```

### Applicable Error Responses

- HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Recharge amount must be greater than zero"
  }
}
```

- HTTP 404 **SESSION\_NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "SESSION_NOT_FOUND",
    "message": "Card session not found"
  }
}
```

**Developer 2 Implementation Notes:** In a transaction: session.balance += amount, create Transaction(type=RECHARGE) and Payment.

## 44. POST /api/v1/card-sessions/:id/purchase

**Purpose:** Executes purchase against active session balance with server-authoritative pricing and inventory deduction.

**Authentication:** `BEARER JWT`

**Role:** `STAFF`, `ORG_ADMIN`

**Permission:** `PURCHASE`

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Scoped to session branch

**Path Parameters:** id: Card Session ID (e.g. SESSION001)

**Query Parameters:** None

**Request Content-Type:** `application/json`

### Request JSON

```
{
  "items": [
    {
      "productId": "PRODUCT001",
      "quantity": 1
    }
  ]
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "transactionId": "TXN001",
    "amount": 120,
    "balance": 380,
    "status": "SUCCESS"
  }
}
```

## Applicable Error Responses

### • HTTP 404 **SESSION\_NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "SESSION_NOT_FOUND",
    "message": "Card session not found"
  }
}
```

### • HTTP 422 **INSUFFICIENT\_BALANCE**:

```
{
  "success": false,
  "error": {
    "code": "INSUFFICIENT_BALANCE",
    "message": "Insufficient session balance"
  }
}
```

### • HTTP 422 **INSUFFICIENT\_INVENTORY**:

```
{
  "success": false,
  "error": {
    "code": "INSUFFICIENT_INVENTORY",
    "message": "Insufficient stock for selected item"
  }
}
```

**Developer 2 Implementation Notes:** Execute in a single \$transaction: fetch product prices from DB, verify balance, deduct inventory, decrement balance, record Transaction.

## 45. POST /api/v1/card-sessions/:id/return

**Purpose:** Settles and closes an active card session, refunds all remaining balance, and resets card to AVAILABLE.

**Authentication:** **BEARER JWT**

**Role:** **STAFF, ORG\_ADMIN**

**Permission:** **CARD\_RETURN**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Scoped to session branch

**Path Parameters:** id: Card Session ID (e.g. SESSION001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "sessionId": "SESSION001",
    "refundedAmount": 380,
    "sessionStatus": "SETTLED",
    "cardStatus": "AVAILABLE"
  }
}
```

## Applicable Error Responses

### • HTTP 404 **SESSION\_NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "SESSION_NOT_FOUND",
    "message": "Card session not found"
  }
}
```

### • HTTP 409 **ALREADY\_SETTLED**:

```
{
  "success": false,
  "error": {
    "code": "ALREADY_SETTLED",
    "message": "Session has already been settled and refunded"
  }
}
```

**Developer 2 Implementation Notes:** In a transaction: record Transaction(type=REFUND, amount=session.balance), set session.balance = 0, session.status = SETTLED, card.status = AVAILABLE.

## 46. GET /api/v1/products

**Purpose:** Lists products for a branch with multi-select category array and live stock quantity. (No tags).

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **PRODUCT\_VIEW**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Scoped to requested branchId

**Path Parameters:** None

**Query Parameters:** branchId (req), category (opt), status (opt), page, limit

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "PRODUCT001",
        "branchId": "branch_001",
        "itemName": "Veg Burger",
        "category": [
          "Veg",
          "Fast Food"
        ],
        "price": 120,
        "quantity": 50,
        "status": "ACTIVE",
        "createdAt": "2026-01-01T00:00:00.000Z",
        "updatedAt": "2026-01-01T00:00:00.000Z"
      },
      {
        "id": "PRODUCT002",
        "branchId": "branch_001",
        "itemName": "Cold Coffee",
        "category": [
          "Beverage"
        ],
        "price": 80,
        "quantity": 100,
        "status": "ACTIVE",
        "createdAt": "2026-01-01T00:00:00.000Z",
        "updatedAt": "2026-01-01T00:00:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 2,
      "totalPages": 1
    }
  }
}
```

## Applicable Error Responses

### • HTTP 401 UNAUTHORIZED:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

### • HTTP 403 BRANCH\_ACCESS\_DENIED:

```
{
  "success": false,
  "error": {
    "code": "BRANCH_ACCESS_DENIED",
    "message": "You do not have access to this branch"
  }
}
```

**Developer 2 Implementation Notes:** category MUST be an array of strings (string[]). tags is removed.

## 47. POST /api/v1/products

**Purpose:** Creates a new product item in a branch catalog with multi-select category array.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, STAFF**

**Permission:** **PRODUCT\_MANAGE**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Scoped to branchId

**Path Parameters:** None**Query Parameters:** None**Request Content-Type:** `application/json`

## Request JSON

```
{
  "branchId": "branch_001",
  "itemName": "Veg Burger",
  "category": [
    "Veg",
    "Fast Food"
  ],
  "price": 120,
  "status": "ACTIVE"
}
```

## Success Response JSON (201 Created)

```
{
  "success": true,
  "data": {
    "id": "PRODUCT001",
    "branchId": "branch_001",
    "itemName": "Veg Burger",
    "category": [
      "Veg",
      "Fast Food"
    ],
    "price": 120,
    "status": "ACTIVE",
    "createdAt": "2026-02-15T12:00:00.000Z",
    "updatedAt": "2026-02-15T12:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 400 `VALIDATION_ERROR`:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Valid product itemName, category selection, and non-negative price are required"
  }
}
```

### • HTTP 403 `BRANCH_ACCESS_DENIED`:

```
{
  "success": false,
  "error": {
    "code": "BRANCH_ACCESS_DENIED",
    "message": "You do not have permission to add products to this branch"
  }
}
```

**Developer 2 Implementation Notes:** Automatically initialize `InventoryItem(productId=product.id, quantity=0)` upon creation.

## 48. GET `/api/v1/inventory`

**Purpose:** Lists current inventory stock quantities for branch products.**Authentication:** `BEARER JWT`**Role:** `SUPER_ADMIN, ORG_ADMIN, STAFF`**Permission:** `INVENTORY_VIEW`**Organization Scope:** Scoped to `req.user.organizationId`**Branch Scope:** Scoped to `requested branchId`**Path Parameters:** None**Query Parameters:** `branchId (req)`, `page`, `limit`**Request Content-Type:** `application/json`

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "INV001",
        "branchId": "branch_001",
        "productId": "PRODUCT001",
        "quantity": 50,
        "updatedAt": "2026-01-01T00:00:00.000Z"
      },
      {
        "id": "INV002",
        "branchId": "branch_001",
        "productId": "PRODUCT002",
        "quantity": 100,
        "updatedAt": "2026-01-01T00:00:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 2,
      "totalPages": 1
    }
  }
}
```

## Applicable Error Responses

### • HTTP 401 UNAUTHORIZED:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

### • HTTP 403 PERMISSION\_DENIED:

```
{
  "success": false,
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "Missing required permission: INVENTORY_VIEW"
  }
}
```

**Developer 2 Implementation Notes:** Query `prisma.inventoryItem.findMany({ where: { branchId } })`.

## 49. PATCH /api/v1/inventory/:id

**Purpose:** Adjusts stock quantity for an inventory record. Prevents negative stock quantities.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, STAFF**

**Permission:** **INVENTORY\_MANAGE**

**Organization Scope:** Scoped to `req.user.organizationId`

**Branch Scope:** Scoped to item branch

**Path Parameters:** `id`: Inventory Record ID (e.g. INV001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "quantity": 100,
  "reason": "Restocked inventory shipment"
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "id": "INV001",
    "branchId": "branch_001",
    "productId": "PRODUCT001",
    "quantity": 100,
    "updatedAt": "2026-02-15T12:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 404 **NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Inventory item not found"
  }
}
```

### • HTTP 422 **INSUFFICIENT\_INVENTORY**:

```
{
  "success": false,
  "error": {
    "code": "INSUFFICIENT_INVENTORY",
    "message": "Inventory stock quantity cannot become negative"
  }
}
```

**Developer 2 Implementation Notes:** Reject any request where quantity < 0.

## 50. GET /api/v1/inventory/import/template

**Purpose:** Downloads standard 3-column CSV import template (itemName,category,price).

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, STAFF**

**Permission:** **INVENTORY\_IMPORT**

**Organization Scope:** None

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "templateCsv": "itemName,category,price\nSample Veg Burger,Veg|Fast Food,120",
    "filename": "inventory_import_template.csv"
  }
}
```

## Applicable Error Responses

- **HTTP 401 UNAUTHORIZED:**

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

**Developer 2 Implementation Notes:** CSV schema contains exactly 3 columns: itemName, category, price. Pipe | delimiter for multi-category.

## 51. POST /api/v1/inventory/import (Preview Stage)

**Purpose:** Validates uploaded CSV content against selected branch, reporting valid/invalid rows and returning atomic previewToken.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, STAFF**

**Permission:** **INVENTORY\_IMPORT**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Scoped to branchId

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

### Request JSON

```
{
  "branchId": "branch_001",
  "csvContent": "itemName,category,price\nVeg Burger,Veg|Fast Food,120\nMasala Chai,Beverage|Hot,30"
}
```

### Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "branchId": "branch_001",
    "totalRows": 2,
    "validRows": [
      {
        "rowNumber": 2,
        "itemName": "Veg Burger",
        "category": [
          "Veg",
          "Fast Food"
        ],
        "price": 120
      },
      {
        "rowNumber": 3,
        "itemName": "Masala Chai",
        "category": [
          "Beverage",
          "Hot"
        ],
        "price": 30
      }
    ],
    "invalidRows": [],
    "createsCount": 1,
    "updatesCount": 1,
    "previewToken": "preview_token_sample_12345"
  }
}
```

## Applicable Error Responses

- **HTTP 400 CSV\_VALIDATION\_ERROR:**

```
{
  "success": false,
```



```

    "error": {
      "code": "CSV_VALIDATION_ERROR",
      "message": "Invalid CSV headers. Required headers: 'itemName', 'category', 'price'"
    }
  }
}

```

• **HTTP 403 ORGANIZATION\_ACCESS\_DENIED:**

```

{
  "success": false,
  "error": {
    "code": "ORGANIZATION_ACCESS_DENIED",
    "message": "Branch does not belong to your organization"
  }
}

```

**Developer 2 Implementation Notes:** Check duplicate item names inside file. Cache valid rows in Redis/DB with previewToken (TTL: 10 mins).

## 52. POST /api/v1/inventory/import (Commit Stage)

**Purpose:** Atomically commits previously validated CSV preview rows to the database.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, STAFF**

**Permission:** **INVENTORY\_IMPORT**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Scoped to branchId

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

### Request JSON

```

{
  "branchId": "branch_001",
  "previewToken": "preview_token_sample_12345",
  "confirm": true
}

```

### Success Response JSON (200 OK)

```

{
  "success": true,
  "data": {
    "importedCount": 2,
    "createsCount": 1,
    "updatesCount": 1
  }
}

```

### Applicable Error Responses

• **HTTP 400 VALIDATION\_ERROR:**

```

{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Preview token has expired or is invalid"
  }
}

```

• **HTTP 403 ORGANIZATION\_ACCESS\_DENIED:**

```

{
  "success": false,
  "error": {
    "code": "ORGANIZATION_ACCESS_DENIED",
    "message": "Branch does not belong to your organization"
  }
}

```

**Developer 2 Implementation Notes:** Wrap all product inserts/updates and inventory creations in a single \$transaction.

## 53. GET /api/v1/analytics

**Purpose:** Retrieves real-time aggregated analytics, branch performance comparison, and hourly peak demand metrics.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **VIEW\_ANALYTICS**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Filterable by branchId

**Path Parameters:** None

**Query Parameters:** branchId (opt), startDate (opt), endDate (opt)

**Request Content-Type:** **application/json**

### Request JSON

\*None (GET Request)\*

### Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "totalTransactions": 1420,
    "totalRechargeVolume": 450000,
    "totalPurchaseVolume": 385000,
    "totalRefundVolume": 65000,
    "activeSessionsCount": 18,
    "activeCardsCount": 250,
    "lowStockItemsCount": 3,
    "branchPerformance": [
      {
        "branchId": "branch_001",
        "branchName": "Main Cafeteria",
        "status": "ACTIVE",
        "transactionCount": 980,
        "purchaseCount": 720,
        "purchaseVolume": 275000,
        "rechargeCount": 260,
        "rechargeVolume": 320000,
        "refundCount": 45,
        "refundVolume": 45000,
        "totalRevenue": 275000,
        "sessionCount": 310,
        "activeSessionsCount": 12,
        "settledSessionsCount": 298,
        "avgTransactionValue": 280.61,
        "avgPurchaseValue": 381.94,
        "productsSoldCount": 1450,
        "inventoryItemCount": 45,
        "lowStockItemCount": 2
      }
    ],
    "peakAnalytics": {
      "busiestHour": "13:00 - 14:00",
      "busiestBranchName": "Main Cafeteria",
      "peakTransactions": 580,
      "offPeakTransactions": 400,
      "peakVolume": 195000,
      "offPeakVolume": 80000
    }
  }
}
```

### Applicable Error Responses

#### • HTTP 401 UNAUTHORIZED:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

- **HTTP 403 PERMISSION\_DENIED:**

```
{
  "success": false,
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "Missing required permission: VIEW_ANALYTICS"
  }
}
```

**Developer 2 Implementation Notes:** Monthly transaction count is for reporting only and never enforces quotas.

## 54. GET /api/v1/analytics/export

**Purpose:** Generates and downloads a compiled PDF report of the active analytics dataset. (PDF only in M0 V10).

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **VIEW\_ANALYTICS**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** Identical filter scope to preview endpoint

**Path Parameters:** None

**Query Parameters:** branchId (opt), format (req: pdf)

**Request Content-Type:** **application/json / application/pdf**

### Request JSON

\*None (GET Request)\*

### Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "filename": "analytics_export_org_001_2026-02-15.pdf",
    "mimeType": "application/pdf",
    "byteLength": 24580,
    "content": "JVBERi0xLjQKJcTl8uXrCjEgMGBvYmoKPDwKL1R5cGUgL0NhdGFsb2cKL1BhZ2VzIDIgMGBSCj4+CmVuZG9iago..."
  }
}
```

### Applicable Error Responses

- **HTTP 400 VALIDATION\_ERROR:**

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Export format must be 'pdf' (M0 Section 20)"
  }
}
```

- **HTTP 403 PERMISSION\_DENIED:**

```
{
  "success": false,
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "Missing required permission: VIEW_ANALYTICS"
  }
}
```

**Developer 2 Implementation Notes:** Set Content-Type: application/pdf header when serving file binary.

## 55. GET /api/v1/reports

**Purpose:** Lists available formal PDF reports for the organization.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **VIEW\_REPORTS**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": "rep_001",
        "title": "Monthly Financial Settlement & Reconciliation Report",
        "type": "FINANCIAL_AUDIT",
        "format": "PDF",
        "generatedAt": "2026-02-01T00:00:00.000Z",
        "downloadUrl": "/api/v1/reports/rep_001/pdf"
      },
      {
        "id": "rep_002",
        "title": "Branch Inventory & Product Movement Summary",
        "type": "INVENTORY_AUDIT",
        "format": "PDF",
        "generatedAt": "2026-02-01T00:00:00.000Z",
        "downloadUrl": "/api/v1/reports/rep_002/pdf"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 2,
      "totalPages": 1
    }
  }
}
```

## Applicable Error Responses

### • HTTP 401 UNAUTHORIZED:

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

### • HTTP 403 PERMISSION\_DENIED:

```
{
  "success": false,
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "Missing required permission: VIEW_REPORTS"
  }
}
```

**Developer 2 Implementation Notes:** Reports in M0 V10 are strictly PDF download only.

## 56. GET /api/v1/reports/:id/pdf

**Purpose:** Downloads the binary PDF file for a formal report.

**Authentication:** **BEARER JWT**

**Role:** **SUPER\_ADMIN, ORG\_ADMIN, STAFF**

**Permission:** **VIEW\_REPORTS**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** None

**Path Parameters:** id: Report ID (e.g. rep\_001)

**Query Parameters:** None

**Request Content-Type:** **application/pdf**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "reportId": "rep_001",
    "filename": "formal_report_rep_001.pdf",
    "mimeType": "application/pdf",
    "content": "JVBERi0xLjQKJcTl8uXrCjEgMCAvYmoKPDwKL1R5cGUgL0NhdGFsb2cKL1BhZ2VzIDIgMCAvZG9iag..."
  }
}
```

## Applicable Error Responses

### • HTTP 403 PERMISSION\_DENIED:

```
{
  "success": false,
  "error": {
    "code": "PERMISSION_DENIED",
    "message": "Missing required permission: VIEW_REPORTS"
  }
}
```

### • HTTP 404 NOT\_FOUND:

```
{
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Report not found"
  }
}
```

**Developer 2 Implementation Notes:** Serve as application/pdf with Content-Disposition: attachment.

## 57. GET /api/v1/subscription/plans

**Purpose:** Lists available subscription tiers for comparison when an Org Admin wishes to request a plan upgrade/downgrade.

**Authentication:** **BEARER JWT**

**Role:** **ORG\_ADMIN, SUPER\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** None

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": [
    {
      "id": "PLAN_BASIC",
      "name": "Basic Plan",
      "status": "ACTIVE",
      "price": 499,
      "currency": "INR",
      "billingInterval": "MONTHLY",
      "branchLimit": 1,
      "staffLimit": 10,
      "cardLimit": 250,
      "inventoryLevel": "Basic",
      "reportsLevel": "Yes",
      "analyticsLevel": "Basic",
      "multiBranchEnabled": false,
      "whiteLabelEnabled": true,
      "supportLevel": "Standard"
    },
    {
      "id": "PLAN_STANDARD",
      "name": "Standard Plan",
      "status": "ACTIVE",
      "price": 1499,
      "currency": "INR",
      "billingInterval": "MONTHLY",
      "branchLimit": 3,
      "staffLimit": 25,
      "cardLimit": 1000,
      "inventoryLevel": "Advanced",
      "reportsLevel": "Yes",
      "analyticsLevel": "Standard",
      "multiBranchEnabled": true,
      "whiteLabelEnabled": true,
      "supportLevel": "Priority"
    },
    {
      "id": "PLAN_PRO",
      "name": "Pro Plan",
      "status": "ACTIVE",
      "price": 2999,
      "currency": "INR",
      "billingInterval": "MONTHLY",
      "branchLimit": 10,
      "staffLimit": 75,
      "cardLimit": 5000,
      "inventoryLevel": "Advanced",
      "reportsLevel": "Yes",
      "analyticsLevel": "Advanced",
      "multiBranchEnabled": true,
      "whiteLabelEnabled": true,
      "supportLevel": "Priority"
    },
    {
      "id": "PLAN_ENTERPRISE",
      "name": "Enterprise Plan",
      "status": "ACTIVE",
      "price": 9999,
      "currency": "INR",
      "billingInterval": "YEARLY",
      "branchLimit": 999,
      "staffLimit": 999,
      "cardLimit": 99999,
      "inventoryLevel": "Advanced",
      "reportsLevel": "Yes",
      "analyticsLevel": "Advanced",
      "multiBranchEnabled": true,
      "whiteLabelEnabled": true,
      "supportLevel": "Dedicated"
    }
  ]
}
```

## Applicable Error Responses

- **HTTP 401 UNAUTHORIZED:**

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}
```

**Developer 2 Implementation Notes:** Filter status = ACTIVE.

## 58. GET /api/v1/subscription

**Purpose:** Retrieves current organization's active subscription, base plan metadata, effective limits, and current usage counts.

**Authentication:** [BEARER JWT](#)

**Role:** [ORG\\_ADMIN](#), [SUPER\\_ADMIN](#)

**Permission:** [NONE](#)

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** [application/json](#)

### Request JSON

\*None (GET Request)\*

### Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "subscription": {
      "id": "sub_001",
      "organizationId": "org_001",
      "planId": "PLAN_STANDARD",
      "status": "ACTIVE",
      "startDate": "2026-01-01T00:00:00.000Z",
      "endDate": "2026-12-31T23:59:59.000Z",
      "renewalDate": "2026-12-31T23:59:59.000Z",
      "paymentStatus": "SUCCESS"
    },
    "plan": {
      "id": "PLAN_STANDARD",
      "name": "Standard Plan",
      "price": 1499,
      "currency": "INR",
      "billingInterval": "MONTHLY",
      "branchLimit": 3,
      "staffLimit": 25,
      "cardLimit": 1000
    },
    "effectiveLimits": {
      "branchLimit": 3,
      "staffLimit": 25,
      "cardLimit": 1000
    },
    "usage": {
      "branchCount": 2,
      "staffCount": 1,
      "cardCount": 2
    }
  }
}
```

### Applicable Error Responses

- **HTTP 401 UNAUTHORIZED:**

```
{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
```

```

    "message": "Authentication required"
  }
}

• HTTP 404 SUBSCRIPTION_NOT_FOUND:
{
  "success": false,
  "error": {
    "code": "SUBSCRIPTION_NOT_FOUND",
    "message": "No subscription record found"
  }
}

```

**Developer 2 Implementation Notes:** Compute active usage counts dynamically by querying branches, staff, and cards.

## 59. GET /api/v1/subscription/payments

**Purpose:** Lists direct subscription invoice payments for the authenticated organization.

**Authentication:** [BEARER JWT](#)

**Role:** [ORG\\_ADMIN](#), [SUPER\\_ADMIN](#)

**Permission:** [NONE](#)

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** page, limit

**Request Content-Type:** [application/json](#)

### Request JSON

\*None (GET Request)\*

### Success Response JSON (200 OK)

```

{
  "success": true,
  "data": {
    "items": [
      {
        "id": "sub_pay_001",
        "subscriptionId": "sub_001",
        "amount": 1499,
        "currency": "INR",
        "status": "SUCCESS",
        "paymentMethod": "DIRECT_BANK_TRANSFER",
        "paymentReference": "DIRECT_NEFT_982341203",
        "createdAt": "2026-01-01T00:00:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 1,
      "totalPages": 1
    }
  }
}

```

### Applicable Error Responses

#### • HTTP 401 UNAUTHORIZED:

```

{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Authentication required"
  }
}

```

**Developer 2 Implementation Notes:** Query `prisma.subscriptionPayment.findMany({ where: { organizationId: req.user.organizationId } })`.



## 60. POST /api/v1/subscription/plan-requests

**Purpose:** Submits a formal plan upgrade or downgrade request to the platform Super Admin.

**Authentication:** **BEARER** **JWT**

**Role:** **ORG\_ADMIN**

**Permission:** **NONE**

**Organization Scope:** Scoped to req.user.organizationId

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

### Request JSON

```
{
  "requestedPlanId": "PLAN_PRO",
  "requestType": "UPGRADE",
  "message": "Need higher staff/card limits for expansion."
}
```

### Success Response JSON (201 Created)

```
{
  "success": true,
  "data": {
    "id": "PCR001",
    "status": "PENDING",
    "requestedPlanId": "PLAN_PRO"
  }
}
```

### Applicable Error Responses

- HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Requested plan ID is required"
  }
}
```

- HTTP 409 **DOWNGRADE\_LIMIT\_EXCEEDED**:

```
{
  "success": false,
  "error": {
    "code": "DOWNGRADE_LIMIT_EXCEEDED",
    "message": "Cannot request downgrade: Current resource usage exceeds target plan limit"
  }
}
```

**Developer 2 Implementation Notes:** If requestType is DOWNGRADE, verify current resource usage does not exceed target plan limits.

## 61. POST /api/v1/public/cards/resolve

**Purpose:** Resolves customer's physical card QR token to determine session eligibility and cafeteria branch info.

**Authentication:** **PUBLIC**

**Role:** **PUBLIC**

**Permission:** **NONE**

**Organization Scope:** Public masked view

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "qrToken": "qr_opaque_hash_card_001"
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "cardDisplayNumber": "MC-***-001",
    "hasActiveSession": true,
    "branchDisplayName": "Main Cafeteria",
    "status": "ACTIVE"
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "QR token is required"
  }
}
```

### • HTTP 404 **CARD\_NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "CARD_NOT_FOUND",
    "message": "Invalid QR code. Card not found."
  }
}
```

### • HTTP 422 **INVALID\_BUSINESS\_STATE**:

```
{
  "success": false,
  "error": {
    "code": "INVALID_BUSINESS_STATE",
    "message": "This card has been blocked. Please visit cafeteria desk."
  }
}
```

**Developer 2 Implementation Notes:** Never return sensitive database IDs or staff credentials. Returns scoped sessionToken.

## 62. POST /api/v1/public/sessions/access

**Purpose:** Generates a dedicated, short-lived portal session JWT from a scanned card QR token.

**Authentication:** **PUBLIC**

**Role:** **PUBLIC**

**Permission:** **NONE**

**Organization Scope:** Public

**Branch Scope:** None

**Path Parameters:** None

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

```
{
  "cardToken": "qr_opaque_hash_card_001"
}
```

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "sessionToken": "portal_jwt_session_token_xyz",
    "expiresAt": "2026-02-15T14:00:00.000Z"
  }
}
```

## Applicable Error Responses

### • HTTP 400 **VALIDATION\_ERROR**:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Card token is required"
  }
}
```

### • HTTP 404 **CARD\_NOT\_FOUND**:

```
{
  "success": false,
  "error": {
    "code": "CARD_NOT_FOUND",
    "message": "Card not found"
  }
}
```

**Developer 2 Implementation Notes:** Sign short-lived portal JWT (TTL: 2 hours) containing sessionId.

## 63. GET /api/v1/public/sessions/:sessionToken

**Purpose:** Retrieves real-time active balance, status, and branch name for customer portal display.

**Authentication:** **PUBLIC (Token in path)**

**Role:** **PUBLIC**

**Permission:** **NONE**

**Organization Scope:** Scoped to session

**Branch Scope:** None

**Path Parameters:** sessionToken: Portal token (e.g. portal\_token\_mc001\_session001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

## Request JSON

\*None (GET Request)\*

## Success Response JSON (200 OK)

```
{
  "success": true,
  "data": {
    "sessionId": "SESSION001",
    "cardDisplayNumber": "MC-***-001",
    "status": "ACTIVE",
    "balance": 380,
    "branchDisplayName": "Main Cafeteria",
    "startedAt": "2026-02-15T12:00:00.000Z",
    "settledAt": null,
    "settlementStatus": null
  }
}
```

## Applicable Error Responses

### • HTTP 401 **UNAUTHORIZED**:

```
{
  "success": false,
  "error": {
```

```

    "code": "UNAUTHORIZED",
    "message": "Invalid portal session token"
  }
}

• HTTP 410 PORTAL_SESSION_EXPIRED:
{
  "success": false,
  "error": {
    "code": "PORTAL_SESSION_EXPIRED",
    "message": "Your session has expired. Please scan your card QR again."
  }
}

```

**Developer 2 Implementation Notes:** Resolve sessionId from sessionToken map/JWT.

## 64. GET /api/v1/public/sessions/:sessionToken/transactions

**Purpose:** Returns transaction history (recharges and purchases) for the customer's current session.

**Authentication:** **PUBLIC (Token in path)**

**Role:** **PUBLIC**

**Permission:** **NONE**

**Organization Scope:** Scoped to session

**Branch Scope:** None

**Path Parameters:** sessionToken: Portal token (e.g. portal\_token\_mc001\_session001)

**Query Parameters:** None

**Request Content-Type:** **application/json**

### Request JSON

\*None (GET Request)\*

### Success Response JSON (200 OK)

```

{
  "success": true,
  "data": {
    "items": [
      {
        "id": "TXN001",
        "type": "RECHARGE",
        "amount": 500,
        "status": "SUCCESS",
        "createdAt": "2026-02-15T12:00:00.000Z"
      },
      {
        "id": "TXN002",
        "type": "PURCHASE",
        "amount": 120,
        "status": "SUCCESS",
        "createdAt": "2026-02-15T12:15:00.000Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 20,
      "total": 2,
      "totalPages": 1
    }
  }
}

```

### Applicable Error Responses

• HTTP 401 UNAUTHORIZED:

```

{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Invalid or expired public portal session token"
  }
}

```

```

}

• HTTP 410 PORTAL_SESSION_EXPIRED:
{
  "success": false,
  "error": {
    "code": "PORTAL_SESSION_EXPIRED",
    "message": "Portal session expired"
  }
}

```

**Developer 2 Implementation Notes:** Query transactions where sessionId = targetSessionId ordered by createdAt ASC.

## 65. GET /api/v1/public/sessions/:sessionToken/receipts

**Purpose:** Returns itemized digital purchase receipts for customer review.

**Authentication:** PUBLIC (Token in path)

**Role:** PUBLIC

**Permission:** NONE

**Organization Scope:** Scoped to session

**Branch Scope:** None

**Path Parameters:** sessionToken: Portal token (e.g. portal\_token\_mc001\_session001)

**Query Parameters:** None

**Request Content-Type:** application/json

### Request JSON

\*None (GET Request)\*

### Success Response JSON (200 OK)

```

{
  "success": true,
  "data": [
    {
      "receiptNumber": "REC_SESSION001_TXN002",
      "transactionId": "TXN002",
      "amount": 120,
      "items": [
        {
          "itemName": "Veg Burger",
          "quantity": 1,
          "price": 120
        }
      ],
      "branchDisplayName": "Main Cafeteria",
      "issuedAt": "2026-02-15T12:15:00.000Z"
    }
  ]
}

```

### Applicable Error Responses

• HTTP 401 UNAUTHORIZED:

```

{
  "success": false,
  "error": {
    "code": "UNAUTHORIZED",
    "message": "Invalid or expired public portal session token"
  }
}

```

• HTTP 410 PORTAL\_SESSION\_EXPIRED:

```

{
  "success": false,
  "error": {
    "code": "PORTAL_SESSION_EXPIRED",
    "message": "Portal session expired"
  }
}

```

**Developer 2 Implementation Notes:** Filter transactions where type = PURCHASE.

---

## 10. CSV / Multipart Contracts

- **Header Row:** Exactly `itemName,category,price`
- **Category Delimiter:** Pipe `|` for multi-select (e.g. `Veg|Fast Food`)
- **Price Format:** Decimal / Integer (e.g. `120` or `120.50`)
- **Two-Phase Import Workflow:**
  1. Client sends CSV content and `branchId` to `POST /api/v1/inventory/import`.
  2. Server performs validation, detects inside-file duplicates, and returns `previewToken` with valid/invalid rows.
  3. Client confirms by calling `POST /api/v1/inventory/import` with `previewToken` and `confirm: true`.
  4. Server applies all row inserts/updates in a single atomic transaction.

## 11. State Machines

### Card Status Lifecycle

- **AVAILABLE** -> Can start a new CardSession (transitions to **ACTIVE**).
- **ACTIVE** -> Card currently assigned to a customer session. Cannot start another session (**409 CARD\_NOT\_AVAILABLE**).
- **BLOCKED** -> Card disabled due to loss/theft. Rejects all issue, recharge, purchase, and settlement attempts.
- **ACTIVE** -> **AVAILABLE** (on session return / settlement).

### Card Session Lifecycle

- **ACTIVE** -> Session in use; accepts recharges and purchases.
- **SETTLED** -> Session closed; remaining balance refunded to customer. Cannot be refunded again (**409 ALREADY\_SETTLED**).

## 12. Authoritative Permission Matrix (20 Frozen Codes)

1. **CARD\_VIEW**, 2. **CARD\_ISSUE**, 3. **CARD\_RETURN**, 4. **CARD\_BLOCK**, 5. **CARD\_UNBLOCK**
6. **RECHARGE**, 7. **PURCHASE**, 8. **REFUND**, 9. **SESSION\_VIEW**
10. **PRODUCT\_VIEW**, 11. **PRODUCT\_MANAGE**, 12. **INVENTORY\_VIEW**, 13. **INVENTORY\_MANAGE**, 14. **INVENTORY\_IMPORT**
15. **VIEW\_ANALYTICS**, 16. **VIEW\_REPORTS**
17. **STAFF\_VIEW**, 18. **STAFF\_MANAGE**
19. **BRANCH\_VIEW**, 20. **BRANCH\_MANAGE**

## 13. Organization & Branch Isolation Rules

- **Super Admin:** Bypass tenant isolation; manages platform entities.
- **Org Admin:** Scoped strictly to `req.user.organizationId`.
- **Staff:** Scoped strictly to `req.user.organizationId` AND `req.user.assignedBranchIds`.
- **Cross-Tenant Violation:** Must reject with **403 ORGANIZATION\_ACCESS\_DENIED**.
- **Cross-Branch Violation:** Must reject with **403 BRANCH\_ACCESS\_DENIED**.

## 14. Financial Calculation & Atomicity Rules

1. **Authoritative Price Lookup:** Client sends `productId` and `quantity`. Backend must ALWAYS query active product price from database.
2. **Balance Verification:** `session.balance >= calculated_total`. If balance insufficient -> **422 INSUFFICIENT\_BALANCE**.
3. **Stock Verification:** `inventory.quantity >= purchase_quantity`. If stock insufficient -> **422 INSUFFICIENT\_INVENTORY**.
4. **Atomic Transaction:** Balance deduction, stock decrement, and **Transaction** record creation MUST execute in a single PostgreSQL `$transaction`.
5. **Double Refund Prevention:** Settle resets `session.balance = 0` and sets `status = 'SETTLED'`. Subsequent return calls return **409 ALREADY\_SETTLED**.

## 15. Idempotency Rules

- **Recharges:** Clients may supply `externalReference` (e.g. manual receipt / UPI ref) to prevent duplicate credits.
- **Purchases:** Ensure unique `transactionId` is generated per purchase.

## 16. Mock API Test Results

- **Total Assertions Executed:** 58 / 58 Passed (100% Success Rate)
- **Automated Test Runner:** `scripts/run-api-contract-tests.ts`
- **Schema Compliance:** 100% Verified against `api-contracts/`

## 17. Postman Testing Requirements (65 Requests)

Developer 2 must import `POSTMAN_COLLECTION.json` into Postman and verify that all **65 requests** execute with HTTP 200/201 success against `http://localhost:4000/api/v1`.

## 18. M0 V10 Contract Gaps & Clarifications

- **Tax & Discounts:** Gross prices inclusive of cafeteria taxes. No separate discount line items in V1.
- **QR vs Barcode:** Both `qrToken` (cryptographic hash) and `physicalCardNumber` resolve to the same Card record.
- **Contract Gaps:** 0 (All rules frozen and clarified per M0 V10).

## 19. Developer 2 Implementation Rules

1. Follow the Prisma schema provided in `DEVELOPER_2_HANDOFF.md`.
2. Mount Express routes under `/api/v1` prefix.
3. Use `bcrypt` for password hashing with salt rounds  $\geq 10$ .
4. Use `jsonwebtoken` for signing JWT access and portal tokens.
5. Ensure all errors are returned in `{ success: false, error: { code, message } }` envelope.

## 20. Final Verification Sign-Off Matrix

| Metric                       | Verification Count | Status            |
|------------------------------|--------------------|-------------------|
| Total M0 Endpoints           | 65                 | ■ Verified        |
| Documented Endpoints         | 65                 | ■ 100% Documented |
| Postman Endpoints            | 65                 | ■ 100% Configured |
| Request JSON Documented      | 65                 | ■ 100% Documented |
| Success Response Documented  | 65                 | ■ 100% Documented |
| Applicable Errors Documented | 126                | ■ Fully Mapped    |
| Missing Endpoints            | 0                  | ■ None            |
| Contract Gaps                | 0                  | ■ Fully Resolved  |